25 Feb 2025

Language Understanding $\lambda(\mu)$ Laboratory

Liquid Neural Networks (LNNs) in Real-World Embedded AI

Dr. Ye Kyaw Thu
Lab Leader, Language Understanding Lab., Myanmar
Visiting Professor, NECTEC, Thailand

Table of Contents

ဒီနေ့ Talk မှာ သုံးမယ့် အတိုကောက် စာလုံးတွေ

1. LNNs (Liquid Neural Networks)
2. ODE (Ordinary Differential Equations)
3. LTC (Liquid Time Constant)
4. NCP (Neural Circuit Policies)
5. CfC (Closed-form Continuous-time)
6. LSTM တို့ CNN စတဲ့ deep learning နဲ့ ပတ်သက်တာတွေကတော့ သိပြီးသားလို့ ယူဆမယ်

AI, ML, Deep learning ကို ထဲထဲဝင်ဝင် သိဖို့အတွက်

1. Linear Algebra (Vectors, Matrix, Dot product, Eigenvalues etc.)
2. Calculus (Derivatives, Partial derivatives, Chain rule, Gradients, Basic integrals etc.)
3. Probability & Statistics (Random variables, Probability distributions, Mean, variance, Conditional probability, Bayes' theorem, Maximum likelihood etc.)
4. Discrete Math (Computer science fundamentals, Graph models, Logic, Transformer & attention mechanisms etc.)
5. Information Theory (Entropy, Cross-entropy, KL divergence, Language models etc.)
6. Optimization (Convex vs non-convex functions, Gradient descent, Stochastic gradient descent, Regularization, Constraints)

A Simple Analogy of AI

- The function of AI can be compared to solving equations, as its purpose is to resolve complex problems. In cases with two variables and two equations, the process is straightforward.

Example:

Equation 1: $\mathbf{x + y = 10}$

Equation 2: $\mathbf{x - y = 2}$

A Simple Analogy of AI

- In your school algebra example (x + 2 = 5), x is the unknown. In AI, we have parameters (often called **weights** and **biases**).
- Imagine a simple AI trying to predict the price of a house. The "equation" might look like this:

$$\text{Price} = (w_1 \times \text{Square Footage}) + (w_2 \times \text{Number of Bedrooms}) + b$$

- The **Square Footage** and **Bedrooms** are the data you provide.
- The **w1**, **w2**, and **b** are the "unknowns" the AI needs to figure out.

A Simple Analogy of AI

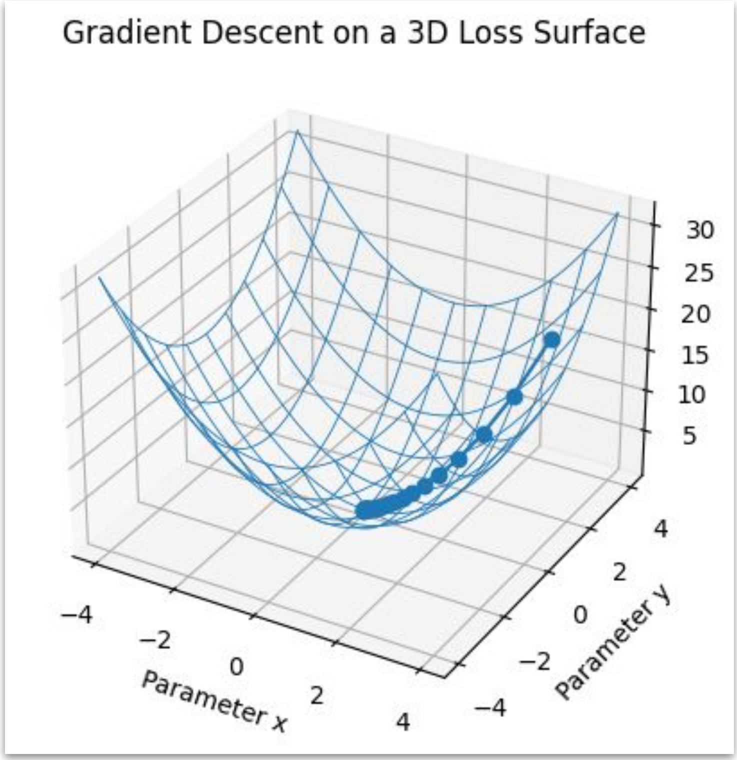
- In algebra, you need at least as many unique equations as you have unknowns to find a single solution. In AI, every piece of data (like a photo of a cat or a sentence in a book) acts like a constraint or an "equation."
- **The Goal**: The AI looks at millions of examples and tries to find values for its "unknowns" (weights) that make the equation work for as many examples as possible.
- **The Difference**: Unlike school math, where there is usually one "perfect" answer, AI looks for the best possible fit that minimizes errors across all the data.

A Simple Analogy of AI

- While our example had two unknowns, a model like GPT-4 has trillions of parameters.
- Instead of a simple line on a 2D graph, the AI is trying to find a "shape" in a space with billions of dimensions.
- It isn't just solving for "price"; it's solving for the probability of the next word in a sentence, the texture of fur in an image, or the tone of a voice.
- We also called "Optimization".

A Simple Analogy of AI

- How the "Solving" Actually Happens?
- We use a process called Gradient Descent.
- Start with a Guess, Check the error (i.e. Loss Function), Adjust the weight, Repeat (billions of times until the equation is accurate as it can get)



Liquid Neural Networks (LNNs)

- Building on the "equation solving" analogy, Liquid Neural Networks (LNNs) are the next evolution. If traditional AI is like solving a static equation, Liquid AI is like solving a living equation.
- Traditional AI as a "line of best fit" on a graph. Once the AI is trained, that line is frozen.
- **Traditional AI (Solid):** Think of it like a concrete sculpture. It’s incredibly detailed, but if you want to change its shape to fit a new room, you have to smash it and build a new one (retraining).
- **Liquid AI (Fluid):** Think of it like a pool of water or a piece of clay. As new information (input) flows in, the "equation" itself ripples and adjusts its shape in real-time. It doesn't just solve the equation once; it keeps solving it as the variables change.

Liquid Neural Networks (LNNs)

- Liquid Neural Networks use Ordinary Differential Equations (ODEs). Instead of the answer being a fixed number, the answer is a description of how things change over time.
- **The Benefit:** Because the math is designed to handle "change," the AI can deal with unexpected events (like a sudden rainstorm while driving or a glitch in a sensor) without getting confused. It just "flows" with the new data.

Liquid Neural Networks (LNNs)

- One of the most mind-blowing facts about LNNs (mentioned in the MIT research by Ramin Hasani and Daniela Rus) is their size.
- **Traditional AI:** A self-driving car model might need 100,000 artificial neurons to stay in its lane.
- **Liquid AI:** Researchers created a Liquid Network that could steer a car using only 19 neurons.
- **How is that possible?** They looked at a tiny worm called C. elegans. It has only 302 neurons, yet it can navigate, find food, and survive in a complex world.

Liquid Neural Networks (LNNs)

- Summary: The “Equation” Comparison

Feature	Traditional AI	Liquid Neural Networks
Math Style	Static Algebra (Fixed weights)	Dynamic Calculus (ODEs)
After Training	Frozen / Fixed	Continues to adapt
Size	Massive (Millions/Billions of nodes)	Tiny (Dozens/Hundreds of nodes)
Best For	Stable data (Text, static images)	Moving data (Video, sensors, robots)

Liquid Neural Networks (LNNs)

- Traditional RNN: the hidden state *h* at step *t* is a simple jump:

$$h_t = \sigma(W x_t + U h_{t-1} + b)$$

- x_t: The current word (e.g., "ကျောင်း").
- h_t−1: The previous memory (e.g., "သူ").
- **The Problem:** The weights ***W*** and ***U*** are fixed. The "internal clock" is fixed to the word count.
- သူ ကျောင်း ? ဆိုတဲ့ ဥပမာ

Liquid Neural Networks (LNNs)

- The hidden state is the solution to a Differential Equation:

$$\frac{dh(t)}{dt} = -\underbrace{\left[\frac{1}{\tau} + f(x(t),\theta)\right]}_{\text{"Liquid" Time Constant}} h(t) + \underbrace{f(x(t),\theta)A}_{\text{Input Target}}$$

- $1/\tau$: The "leakage." How fast the neuron forgets.
- *f(x,θ)*: This is the "Liquid" part. The input word ("ကျောင်း") actually changes the speed of the neuron's reaction.
- **Continuous Time:** We can calculate *h(t)* at any moment, even halfway between words.

Liquid Neural Networks (LNNs)

- Visualizing the "invisible math" happening inside two different networks

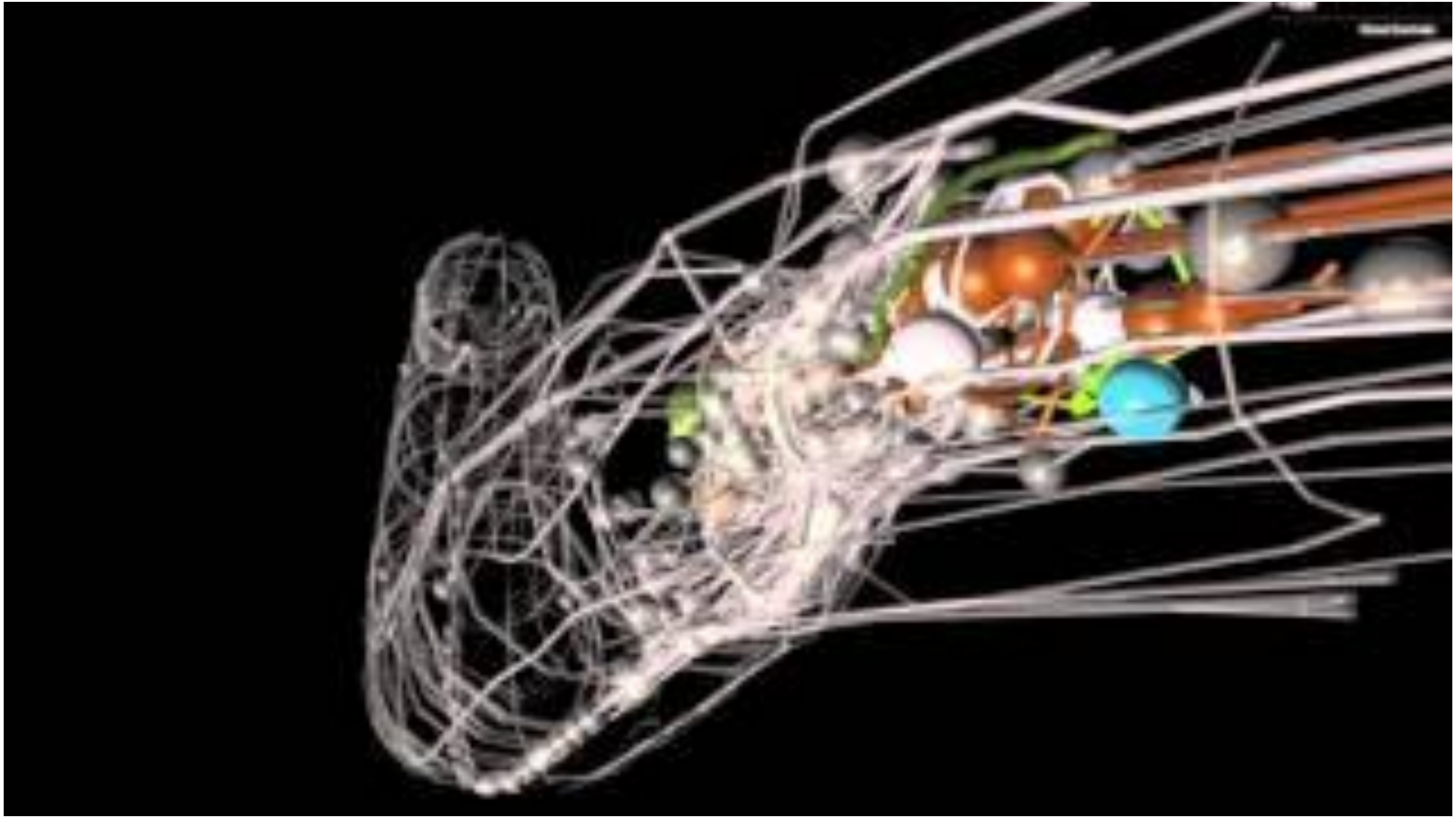
Sources: Wikipedia

Liquid Neural Networks (LNNs)

Concept	Digital Clock (Box)	Hourglass (Liquid)
Logic	"If it's Time 1, do Task A. If it's Time 2, do Task B."	"How much has changed since the last time I looked?"
Math	Algebra (y=mx+b)	Calculus (dt/dy=flow)

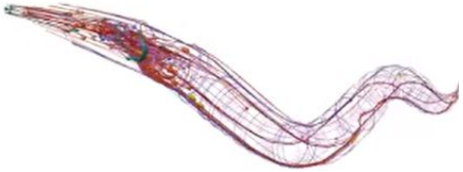
- In our analogy, the **Hourglass is the Neuron**,
- the **Sand is the information** (the state)
- the **Weather is the Input Data**.

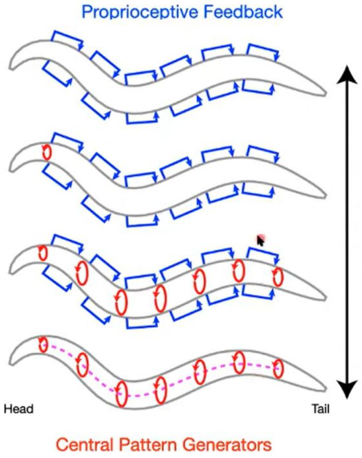
The Evolution of LNNs: From Worms to ODEs

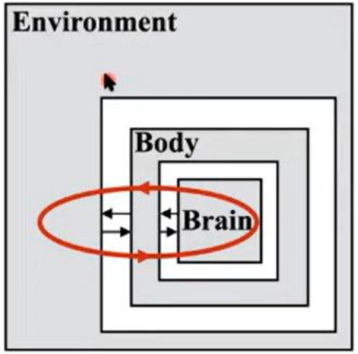


The Evolution of LNNs: From Worms to ODEs

- Microscopic roundworm (nematode), ~1 mm long
- Found in temperate soil environments worldwide
- Feeds on bacteria; in labs, fed E. coli
- Transparent body enables direct observation of cells



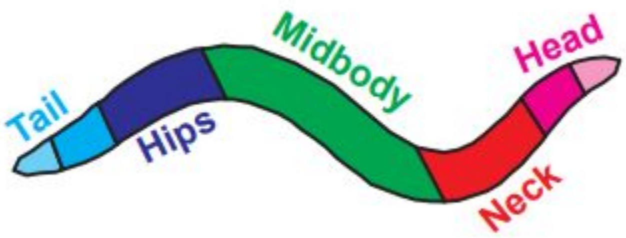




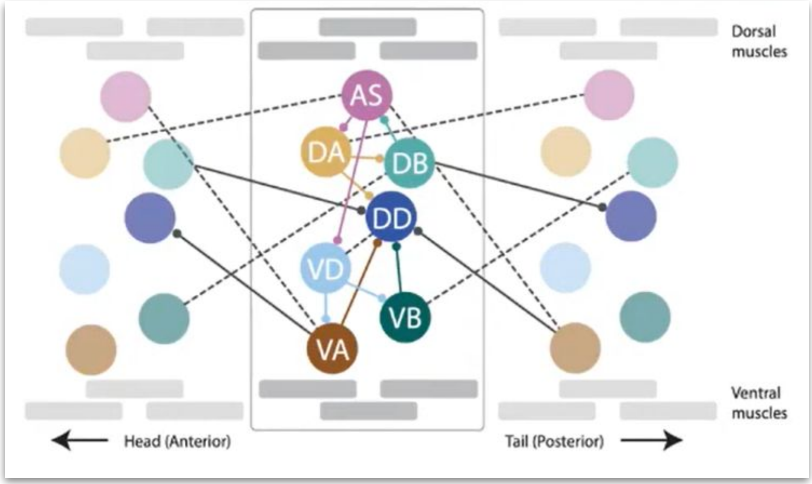
Sources: https://www.youtube.com/watch?v=lXrtoqu4Uso

The Evolution of LNNs: From Worms to ODEs

- Two forms: hermaphrodite (959 cells) and male (1,031 cells)
- Nervous system contains only 302 neurons
- Entire neural connectome fully mapped
- Extremely simple compared to the human brain



Sources: https://www2.mrc-lmb.cam.ac.uk/groups/wschafer/wp-content/uploads/sites/38/tjucikas_phd_thesis.pdf



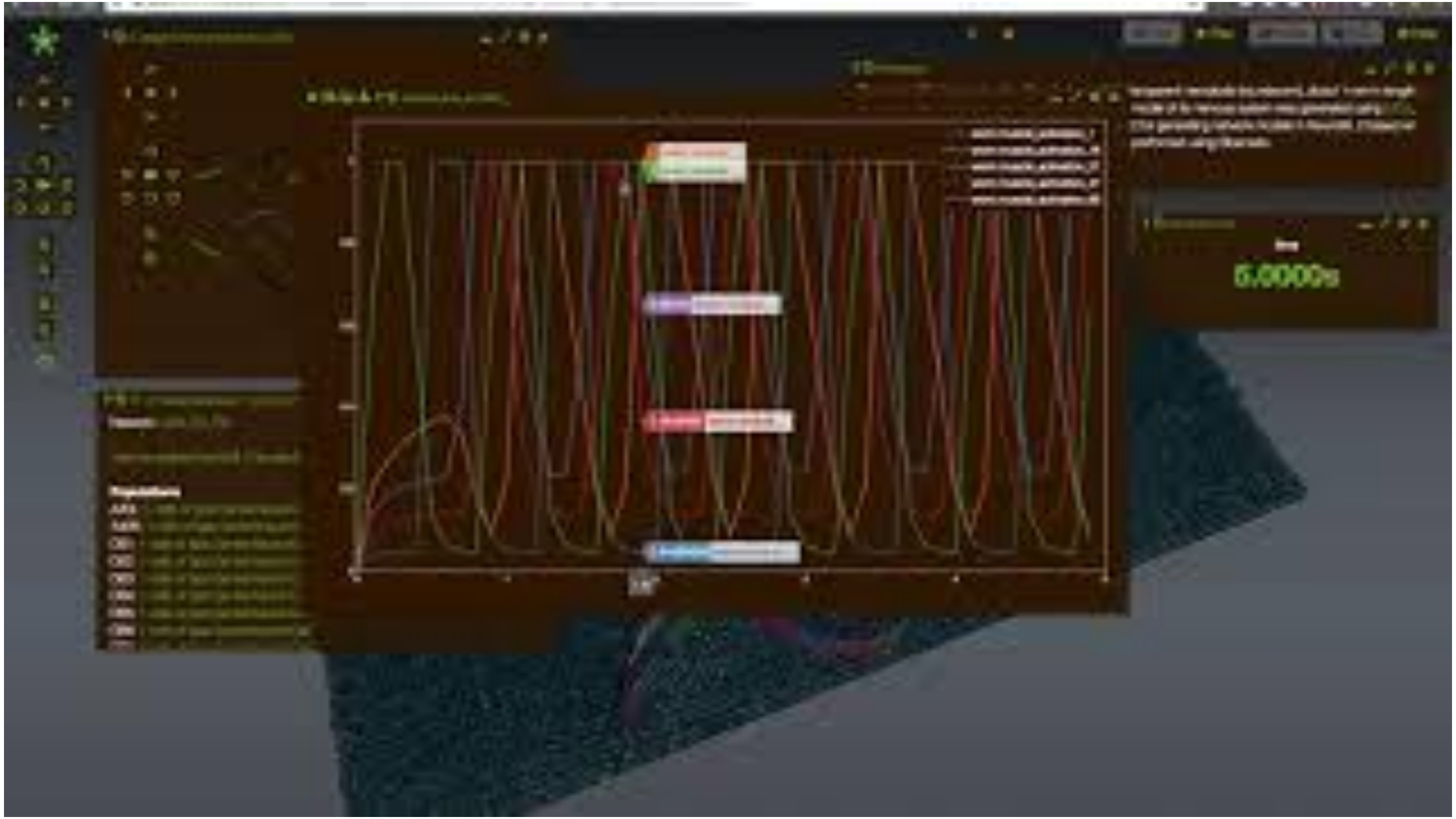
Sources: https://www.youtube.com/watch?v=IXrtoqu4Uso

The Evolution of LNNs: From Worms to ODEs

- Primitive behaviors: feeding, locomotion, reproduction
- Complex behaviors: learning, mating, social interaction
- Behavior reflects nervous system activity
- Powerful model organism for studying biology and disease

Sources: https://takeishilab.com/

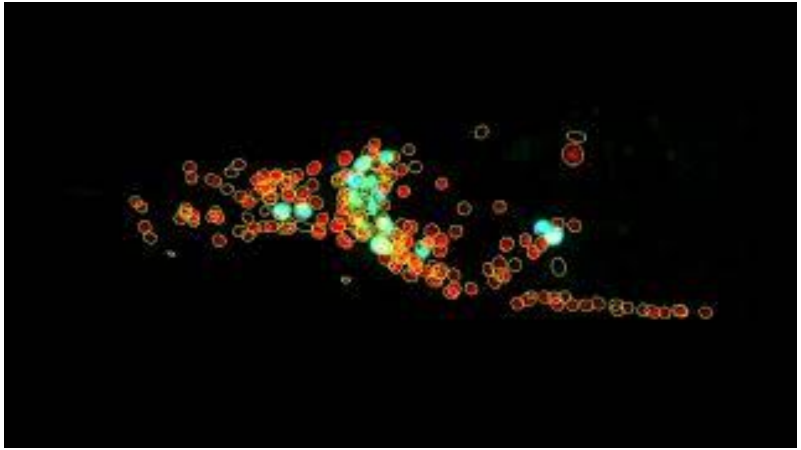
The Evolution of LNNs: From Worms to ODEs



Sources: https://www.youtube.com/watch?v=M0rpT831ngM

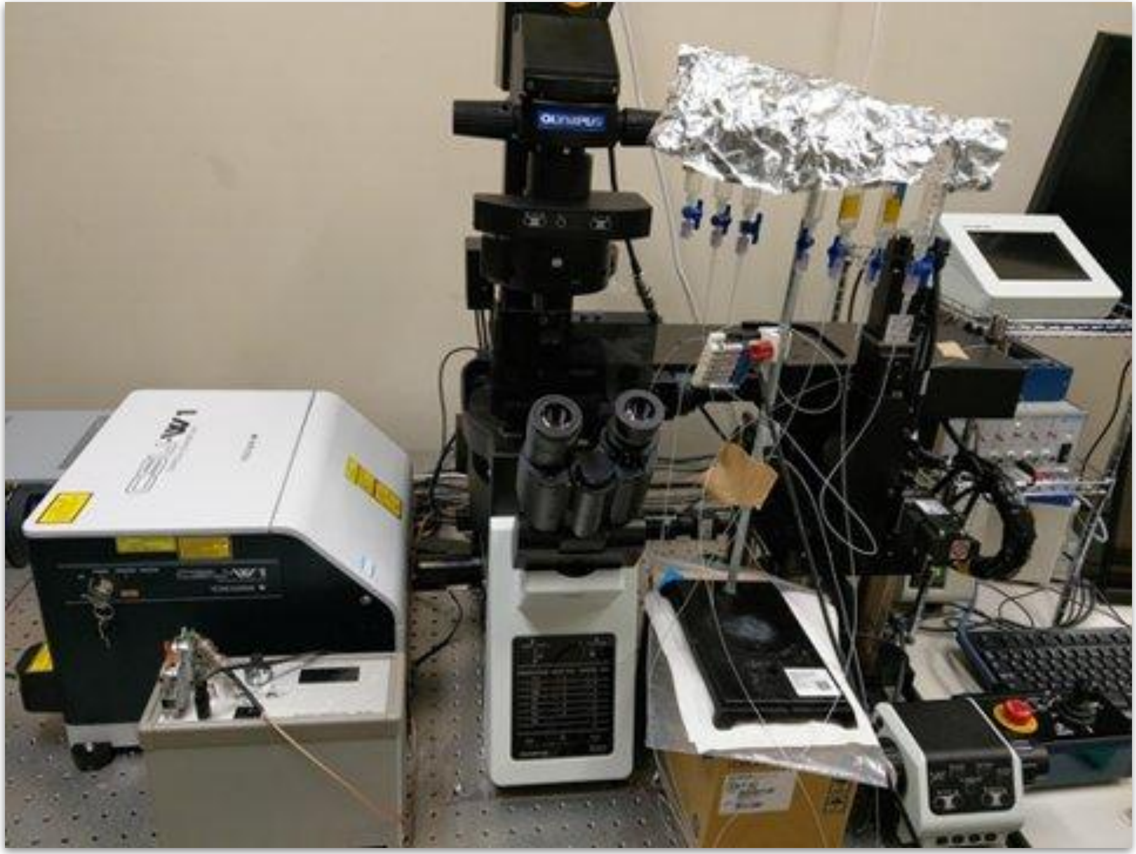
The Evolution of LNNs: From Worms to ODEs

- Whole-brain, single-cell–resolution imaging is key to understanding information processing
- C. elegans is ideal due to its small, transparent body
- Neural activity is recorded in living worms using fluorescence microscopy and microfluidics
- A 4D microscope enables simultaneous observation of all head neurons
- Image analysis and cell identification map neural activity to known circuits



Sources:
https://www.youtube.com/watch?v=ycJ-vcBdEXU&t=19s

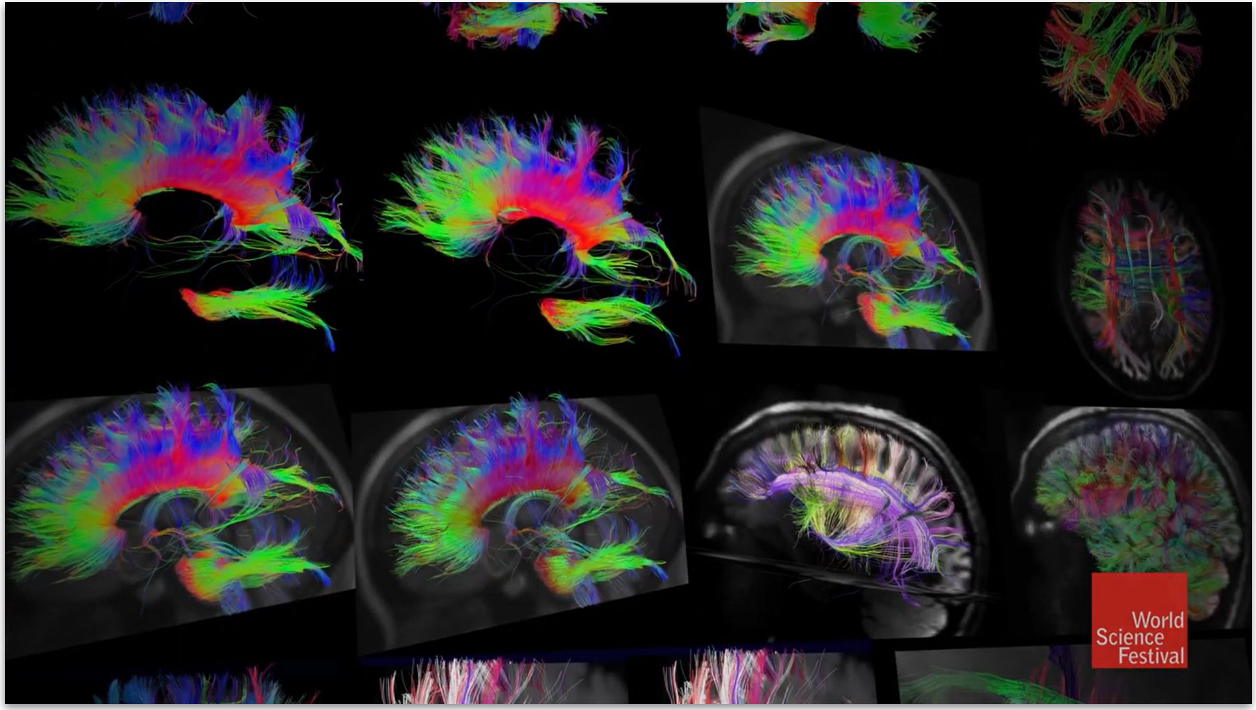
The Evolution of LNNs: From Worms to ODEs



- 4D Microscope (i.e 3D space + time)

Sources: https://www.bs.s.u-tokyo.ac.jp/~toyoshimalab/en/research/index.html

The Evolution of LNNs: From Worms to ODEs



- Structural connections
- Functional connections
- a 1 millimeter cube of brain tissue contains 50,000 nerve cells and about 1 billion synapses
- Currently, no way to model an entire human brain by diagramming its micro wiring

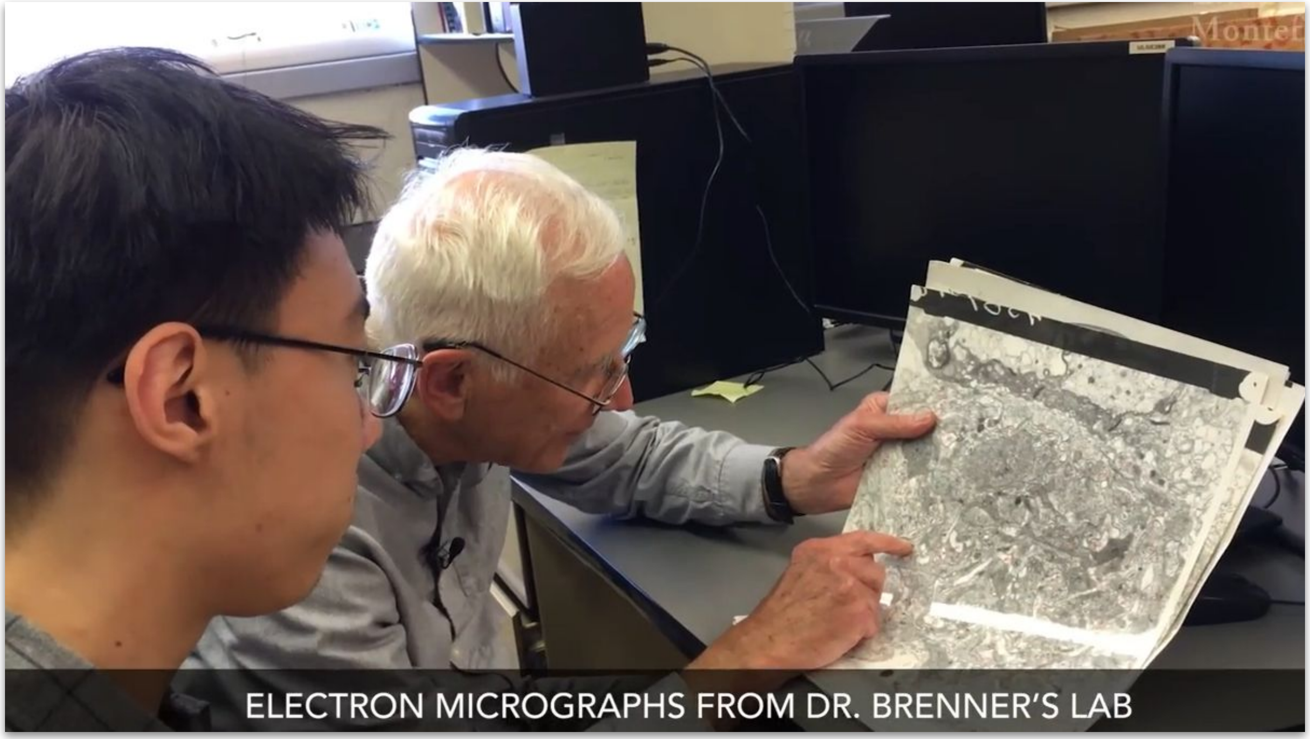
The Evolution of LNNs: From Worms to ODEs

Volume 571 Issue 7763, 4 July 2019

- Knowing how the components of a nervous system are connected together is crucial for understanding how that system works. In this week's issue, Scott Emmons and his colleagues present complete wiring diagrams for both sexes of the model organism Caenorhabditis elegans. These neural maps, or connectomes, which update an influential 1986 publication, were built up from both new and previously published electron micrographs and encompass all connections in the nematode from sensory input to end-organ output. The maps have allowed the researchers to determine the location of each synapse and to assign an indirect measure of the strength of each connection based on the physical size of its constituent synapses.

Sources: https://www.nature.com/nature/volumes/571/issues/7763

The Evolution of LNNs: From Worms to ODEs

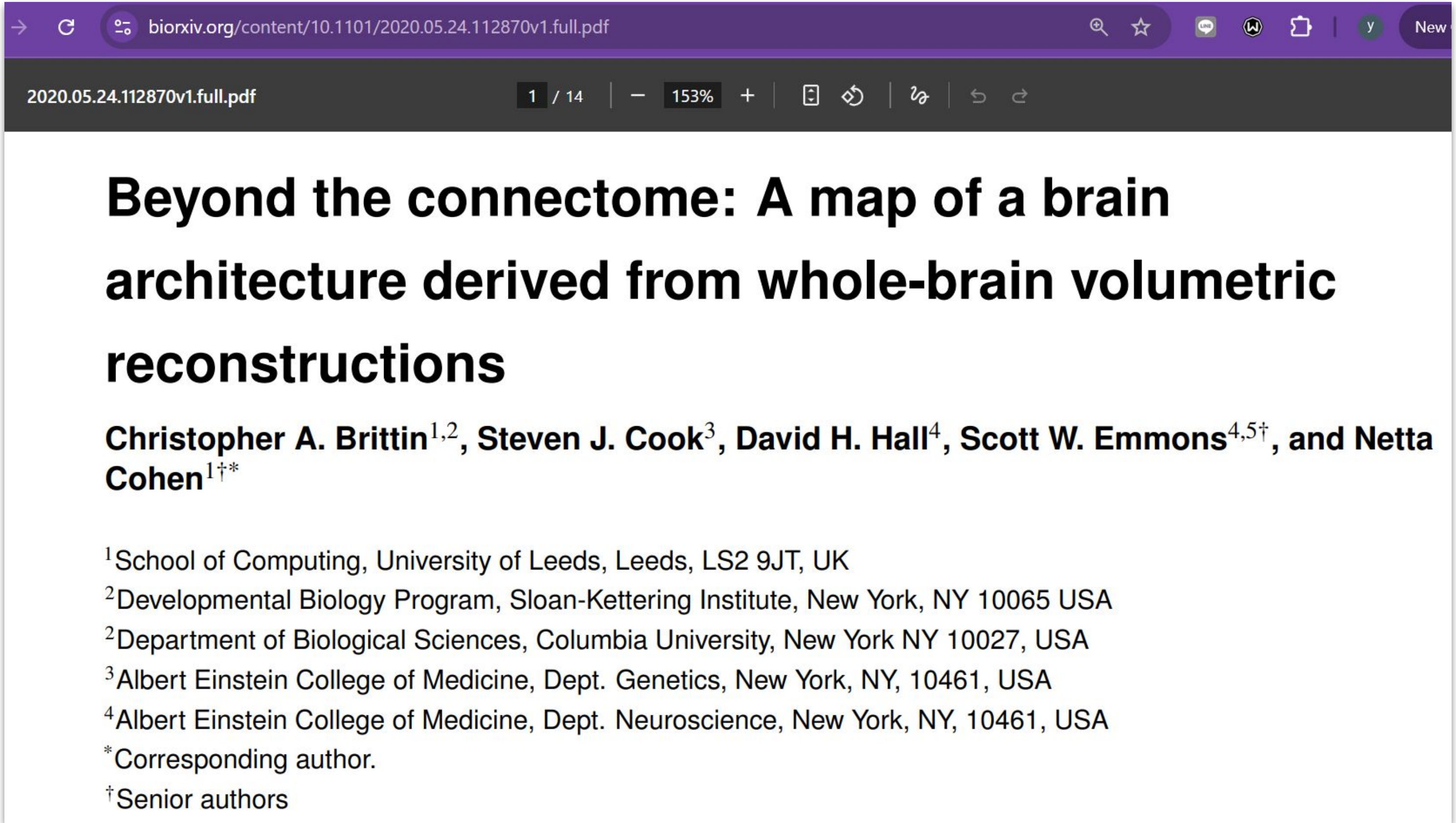


- The role of C. Elegans

Sources: [*https://www.youtube.com/watch?v=LBKEyGaPamY*](https://www.youtube.com/watch?v=LBKEyGaPamY)

- Dr. Scott Emmons and colleagues continued analyzing

The Evolution of LNNs: From Worms to ODEs

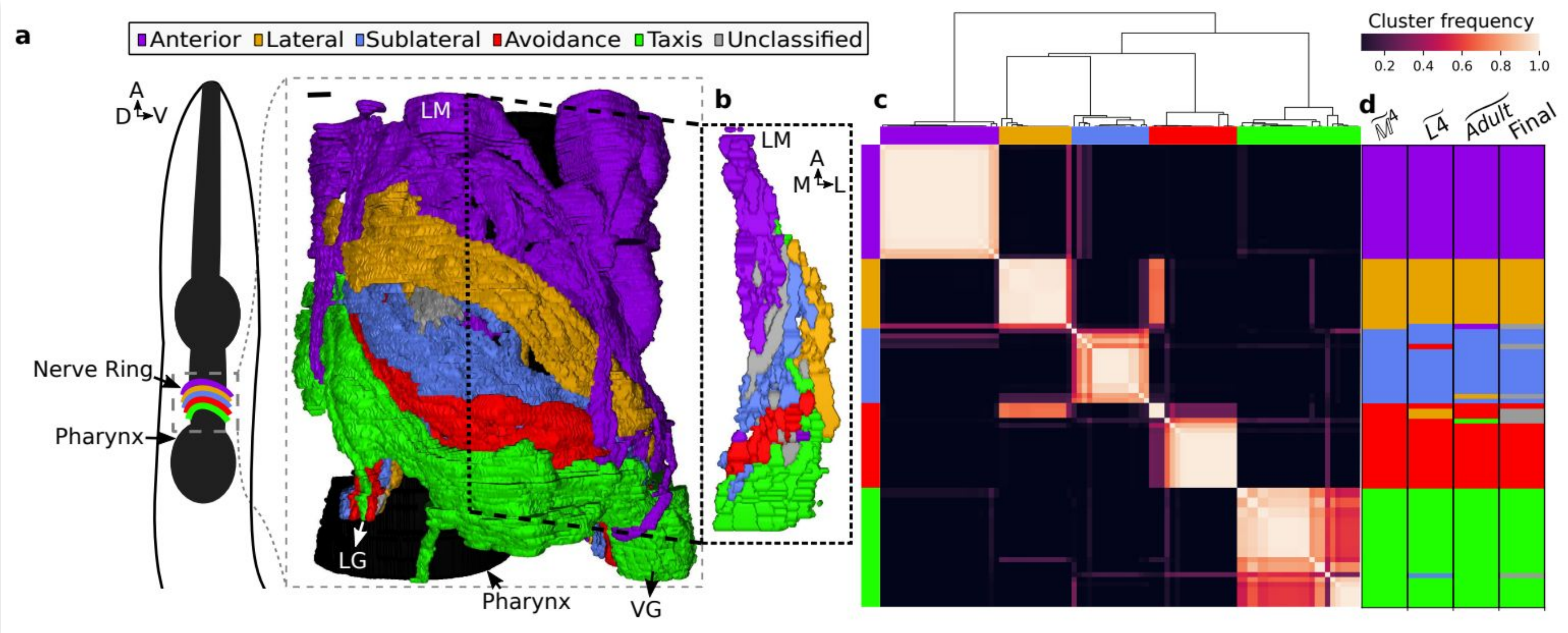


Beyond the connectome: A map of a brain architecture derived from whole-brain volumetric reconstructions

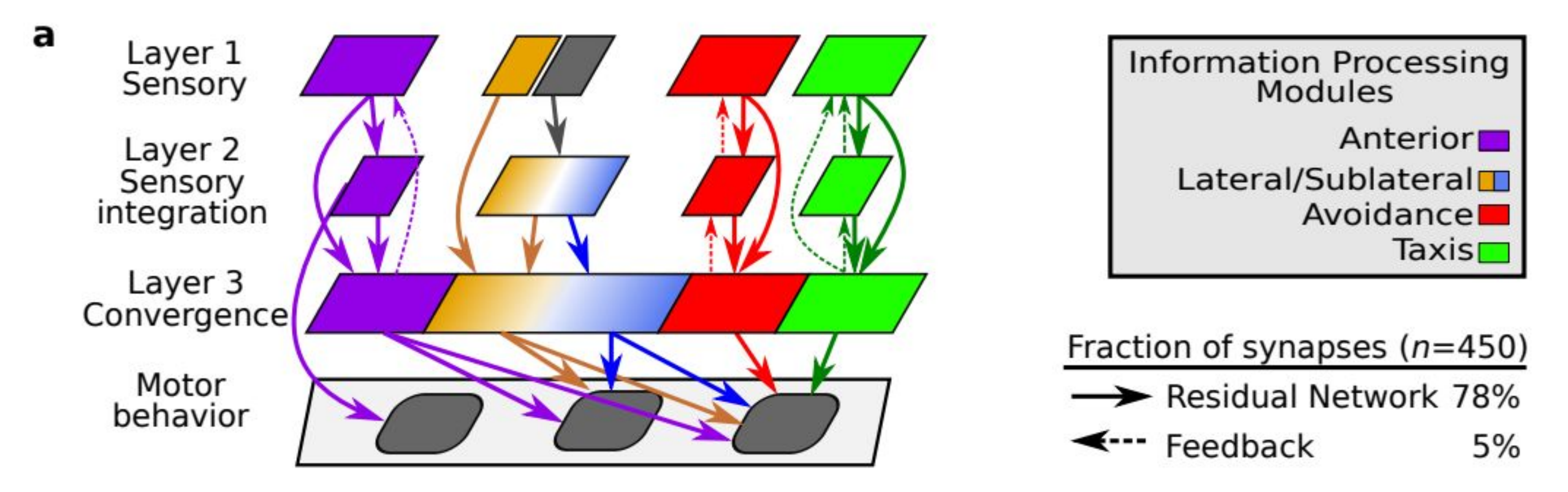
Christopher A. Brittin[1,2], Steven J. Cook[3], David H. Hall[4], Scott W. Emmons[4,5†], and Netta Cohen[1†*]

[1]School of Computing, University of Leeds, Leeds, LS2 9JT, UK
[2]Developmental Biology Program, Sloan-Kettering Institute, New York, NY 10065 USA
[2]Department of Biological Sciences, Columbia University, New York NY 10027, USA
[3]Albert Einstein College of Medicine, Dept. Genetics, New York, NY, 10461, USA
[4]Albert Einstein College of Medicine, Dept. Neuroscience, New York, NY, 10461, USA
[*]Corresponding author.
[†]Senior authors

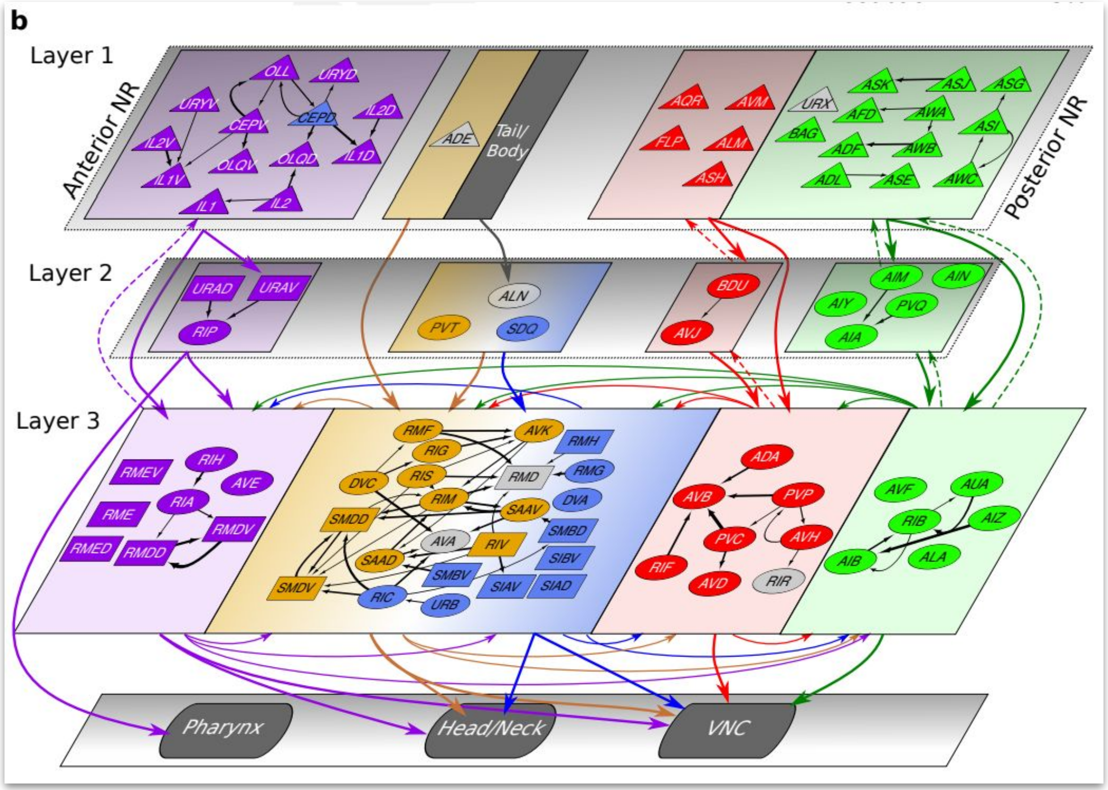
The Evolution of LNNs: From Worms to ODEs



The Evolution of LNNs: From Worms to ODEs

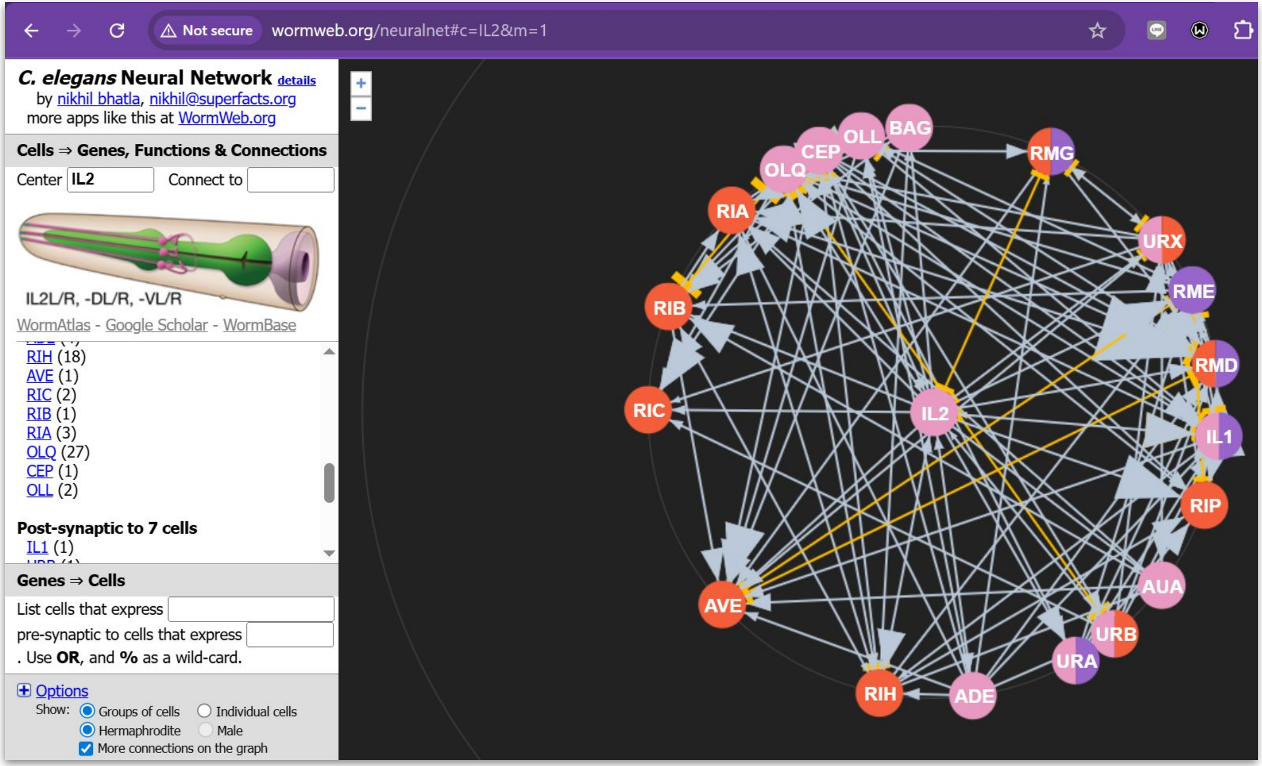


The Evolution of LNNs: From Worms to ODEs



- Every single neuron has a unique 2-to-4-letter name
- For example, RMEV: Ring Motor Neuron, Efferent, Ventral, RIH: Ring Interneuron, Head
- Triangles: sensory neurons that receive input from the environment (touch, chemicals, light)
- Ovals: interneurons (processing units)
- Rectangles: motoneurons

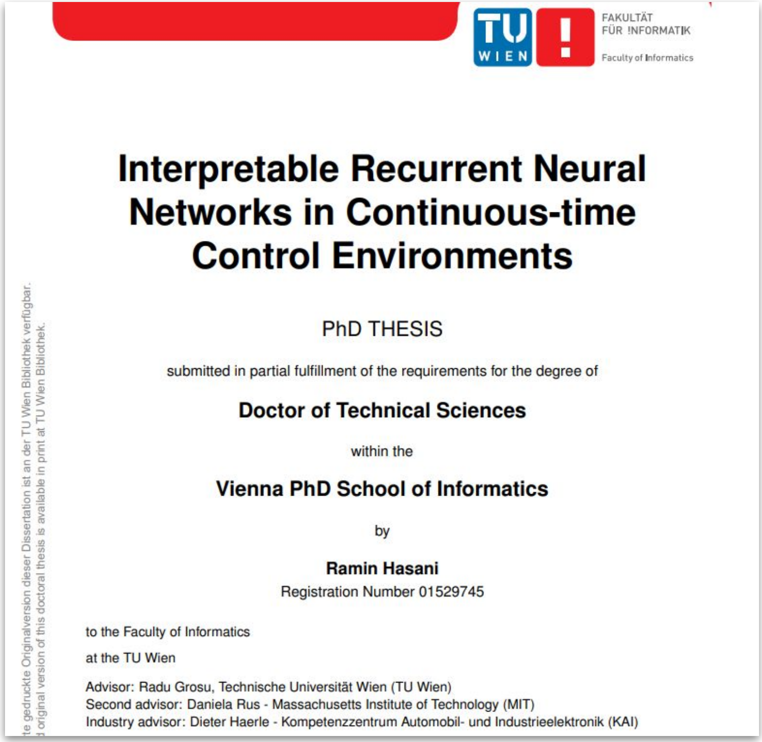
The Evolution of LNNs: From Worms to ODEs



- **PVQ:** Posterior Ventral Q-cell (Interneuron); handles sensory integration and sexual circuits.
- **AVF:** Anterior Ventral F-cell (Interneuron); involved in coordinating locomotion and egg-laying.

Sources: http://wormweb.org/neuralnet#c=IL2&m=1

The Evolution of LNNs: From Worms to ODEs



FAKULTÄT FÜR INFORMATIK
Faculty of Informatics

Interpretable Recurrent Neural Networks in Continuous-time Control Environments

PhD THESIS

submitted in partial fulfillment of the requirements for the degree of

Doctor of Technical Sciences

within the

Vienna PhD School of Informatics

by

Ramin Hasani
Registration Number 01529745

to the Faculty of Informatics

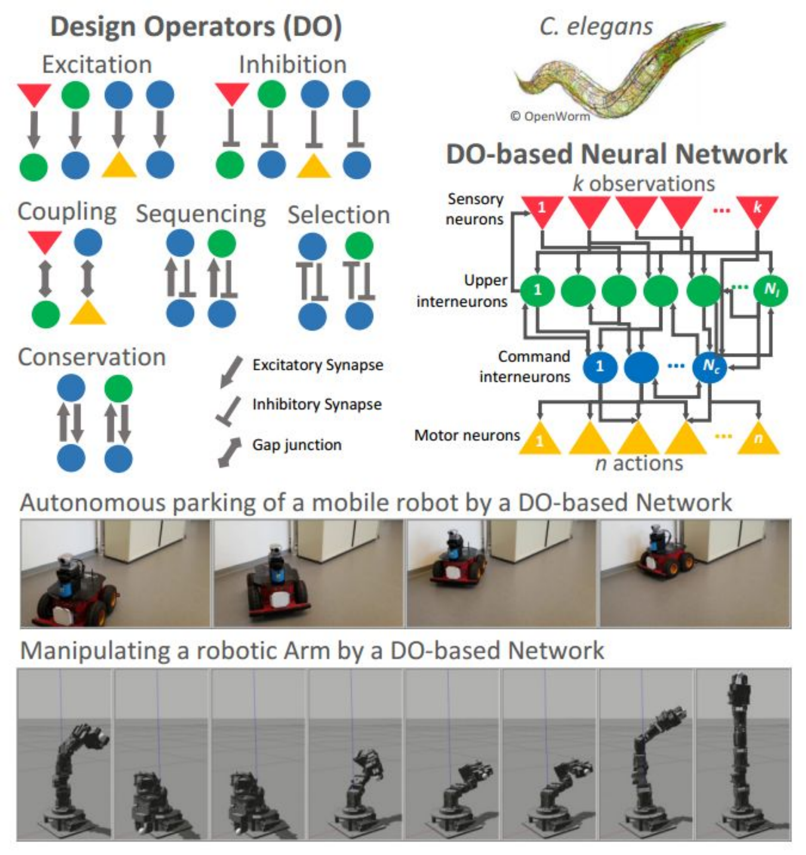
at the TU Wien

Advisor: Radu Grosu, Technische Universität Wien (TU Wien)
Second advisor: Daniela Rus - Massachusetts Institute of Technology (MIT)
Industry advisor: Dieter Haerle - Kompetenzzentrum Automobil- und Industrieelektronik (KAI)

te gedruckte Originalversion dieser Dissertation ist an der TU Wien Bibliothek verfügbar.
d original version of this doctoral thesis is available in print at TU Wien Bibliothek.

- We invented liquid neural networks, a class of brain-inspired systems that can stay adaptable and robust to changes even after training [R. Hasani, PhD Thesis] [Lechner et al. Nature MI, 2020] [pdf] (2016-2020).
- We then analytically and experimentally showed they are universal approximators [Hasani et al. AAAI, 2021], expressive continuous-time machine learning systems for sequential data [Hasani et al. AAAI, 2021] [Hasani et al. Nature MI, 2022], parameter efficient in learning new skills [Lechner et al. Nature MI, 2020] [pdf], causal and interpretable [Vorbach et al. NeurIPS, 2021] [Chahine et al. Science Robotics 2023] [pdf], and when linearized they can efficiently model very long-term dependencies in sequential data [Hasani et al. ICLR 2023].

The Evolution of LNNs: From Worms to ODEs



- We design compact, interpretable and noise-robust neural controllers inspired by the relational structures of the worm's brain, to control robots
- Experimental demonstrations of the superiority of the performance of DO-based networks in terms of their compactness, robustness to noise, and their interpretable dynamics, compared to contemporary RNN architectures.

The Evolution of LNNs: From Worms to ODEs

Liquid Time-Constant Networks

Ramin Hasani,[1,3*] **Mathias Lechner,**[2*] **Alexander Amini,**[1] **Daniela Rus,**[1] **Radu Grosu**[3]

[1] Massachusetts Institute of Technology (MIT)
[2] Institute of Science and Technology Austria (IST Austria)
[3] Technische Universitaẗ Wien (TU Wien)
rhasani@mit.edu, mathias.lechner@ist.ac.at, amini@mit.edu, rus@csail.mit.edu, radu.grosu@tuwien.ac.at

Abstract

We introduce a new class of time-continuous recurrent neural network models. Instead of declaring a learning system's dynamics by implicit nonlinearities, we construct networks of linear first-order dynamical systems modulated via nonlinear interlinked gates. The resulting models represent dynamical systems with varying (i.e., *liquid*) time-constants coupled to their hidden state, with outputs being computed by numerical differential equation solvers. These neural networks exhibit stable and bounded behavior, yield superior expressivity within the family of neural ordinary differential equations,

by the following equation (Funahashi and Nakamura 1993): $\frac{d\mathbf{x}(t)}{dt} = -\frac{\mathbf{x}(t)}{\tau} + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)$, in which the term $-\frac{\mathbf{x}(t)}{\tau}$ assists the autonomous system to reach an equilibrium state with a time-constant τ. $\mathbf{x}(t)$ is the hidden state, $\mathbf{I}(t)$ is the input, t represents time, and f is parametrized by θ.

We propose an alternative formulation: let the hidden state flow of a network be declared by a system of linear ODEs of the form: $d\mathbf{x}(t)/dt = -\mathbf{x}(t)/\tau + \mathbf{S}(t)$, and let $\mathbf{S}(t) \in \mathbb{R}^M$ represent the following nonlinearity determined by $\mathbf{S}(t) = f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)(A - \mathbf{x}(t))$, with parameters θ and A. Then, by plugging in $\mathbf{S}$ into the hidden states equation, we get:

The Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI-21)

The Evolution of LNNs: From Worms to ODEs

nature machine intelligence

Explore content ⌄ About the journal ⌄ Publish with us ⌄

nature > nature machine intelligence > articles > article

Article | Open access | Published: 15 November 2022

Closed-form continuous-time neural networks

Ramin Hasani, Mathias Lechner, Alexander Amini, Lucas Liebenwein, Aaron Ray, Max Tschaikowski, Gerald Teschl & Daniela Rus

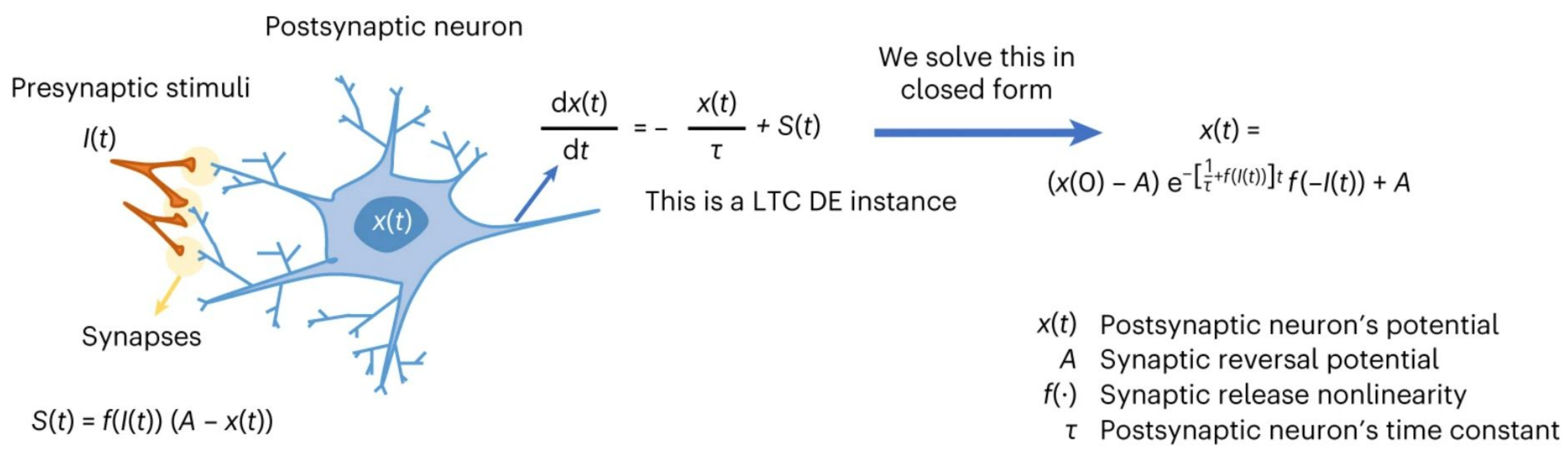
Nature Machine Intelligence **4**, 992–1003 (2022) | Cite this article

A Publisher Correction to this article was published on 29 November 2022

- နောက်ပိုင်း LNNs မော်ဒယ်တွေအားလုံးက ဒီ CfC နည်းလမ်း ကိုပဲ အခြေခံတယ်

Sources: https://www.nature.com/articles/s42256-022-00556-7

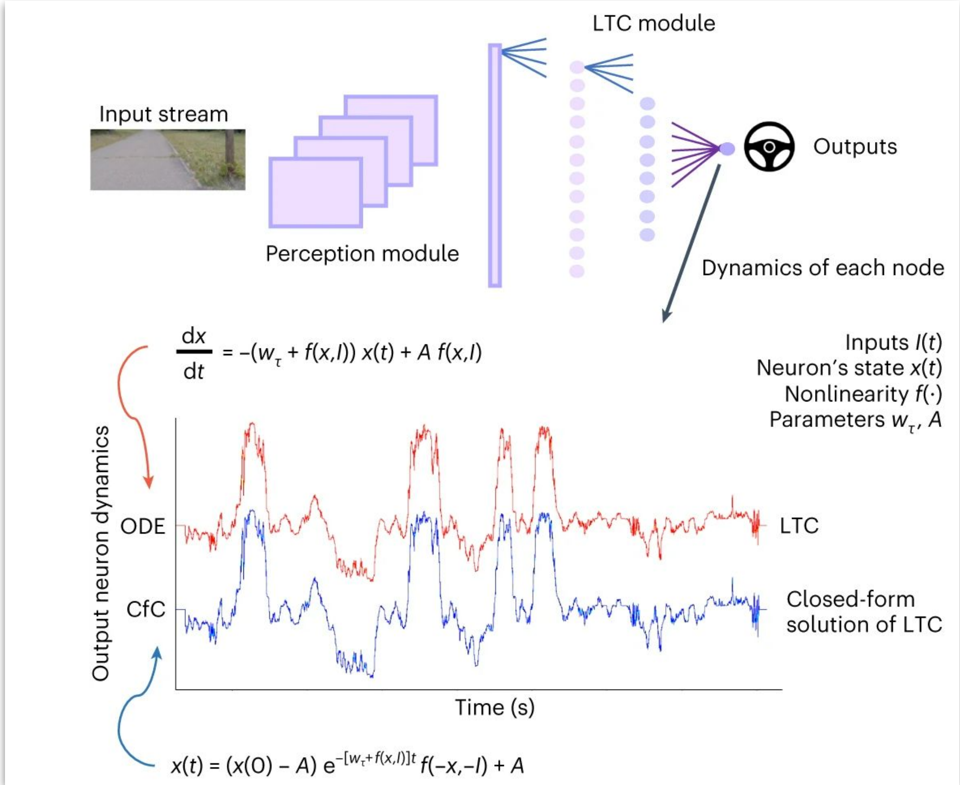
The Evolution of LNNs: From Worms to ODEs



A postsynaptic neuron receives the stimuli $I(t)$ through a nonlinear conductance-based synapse model. Here, $S(t)$ stands for the synaptic current. The dynamics of the membrane potential of this postsynaptic neuron are given by the DE presented in the middle. This equation is a fundamental building block of LTC networks[1], for which there is no known closed-form expression. Here, we provide an approximate solution for this equation which shows the interaction of nonlinear synapses with postsynaptic neurons in closed form.

Source: https://www.nature.com/articles/s42256-022-00556-7/figures/1

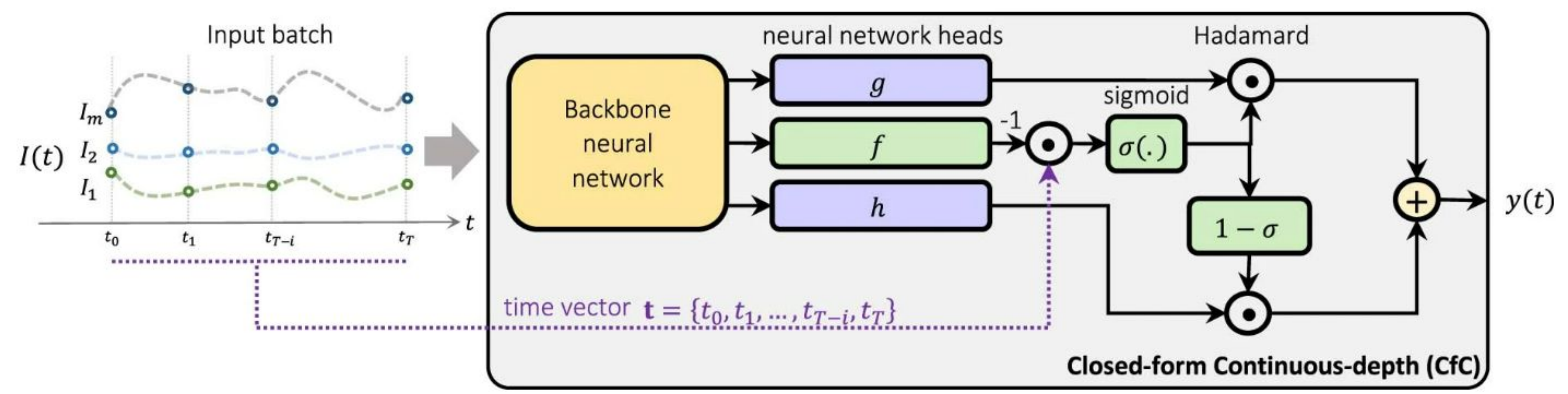
The Evolution of LNNs: From Worms to ODEs



- ဒီ graph ရလဒ် ကနေပြီး CfC နဲ့ ODE ရလဒ်က ဘယ်လောက်ထိ နီးစပ်သလဲ ဆိုတာကို မြင်ရပါလိမ့်မယ်

https://www.nature.com/articles/s42256-022-00556-7/figures/2

The Evolution of LNNs: From Worms to ODEs



A backbone neural network layer delivers the input signals into three head networks g, f and h. f acts as a *liquid* time-constant for the sigmoidal time-gates of the network. g and h construct the nonlinearities of the overall CfC network.

Source: https://www.nature.com/articles/s42256-022-00556-7/figures/4

- ဒီပုံကတော့ လက်တွေ့ implementation/training လုပ်တဲ့အခါမှာ ဘယ်လိုလုပ်သလဲ ဆိုတာကို ရှင်းပြတဲ့ ပုံပါ။

The Evolution of LNNs: From Worms to ODEs

Ramin Hasani • Following
Co-founder & CEO @ Liquid AI
2yr •

United States Air Force successfully flew an X-62 VISTA jet aircraft (derived from the F-16D Fighting Falcon) autonomously by liquid neural networks!

This is an important step towards achieving explainable, safe, and robust autonomy at scale!

So proud of my co-inventors and teammates Mathias Lechner, Tsun-Hsuan Wang, Alexander Amini, and Daniela Rus at the Massachusetts Institute of Technology and Liquid AI!

Hell of a job by our colleagues at the MIT Lincoln Laboratory, AI Accelerator, Defense Advanced Research Projects Agency (DARPA), and 412th Test Wing, Edwards Air Force Base! Special thanks to the amazing Josh "Voodoo" Rountree for leading the effort and trusting our technology!

Sources: *linkedin.com*

The Evolution of LNNs: From Worms to ODEs

Sources: linkedin.com

The Evolution of LNNs: From Worms to ODEs

SCIENCE ROBOTICS | RESEARCH ARTICLE

ARTIFICIAL INTELLIGENCE

Robust flight navigation out of distribution with liquid neural networks

Makram Chahine†, Ramin Hasani†*, Patrick Kao†, Aaron Ray†, Ryan Shubert, Mathias Lechner, Alexander Amini, Daniela Rus

Autonomous robots can learn to perform visual navigation tasks from offline human demonstrations and generalize well to online and unseen scenarios within the same environment they have been trained on. It is challenging for these agents to take a step further and robustly generalize to new environments with drastic scenery changes that they have never encountered. Here, we present a method to create robust flight navigation agents that successfully perform vision-based fly-to-target tasks beyond their training environment under drastic distribution shifts. To this end, we designed an imitation learning framework using liquid neural networks, a brain-inspired class of continuous-time neural models that are causal and adapt to changing conditions. We observed that liquid agents learn to distill the task they are given from visual inputs and drop irrelevant features. Thus, their learned navigation skills transferred to new environments. When compared with several other state-of-the-art deep agents, experiments showed that this level of robustness in decision-making is exclusive to liquid networks, both in their differential equation and closed-form representations.

- OOD ကို လက်တွေ့မှာ မော်ဒယ်တော်တော်များများ က fail ဖြစ်

Sources:
https://cap.csail.mit.edu/sites/default/files/research-pdfs/Robust%20flight%20navigation%20out%20of%20distribution%20with%20liquid%20neural%20networks.pdf

The Evolution of LNNs: From Worms to ODEs

- A: Task - Visual navigation to target

The Evolution of LNNs: From Worms to ODEs

- B: Test - Robust task understanding under heavy distribution shift

The Evolution of LNNs: From Worms to ODEs

- C: Samples from train set (drone view)

The Evolution of LNNs: From Worms to ODEs

- D: Samples from the four test environments (drone view)

The Evolution of LNNs: From Worms to ODEs

Table 1. Closed-loop evaluation of the fly-to-target task for the trained policies in four testing environments. Quantitative results are success rates. Higher is better. (N = 40)

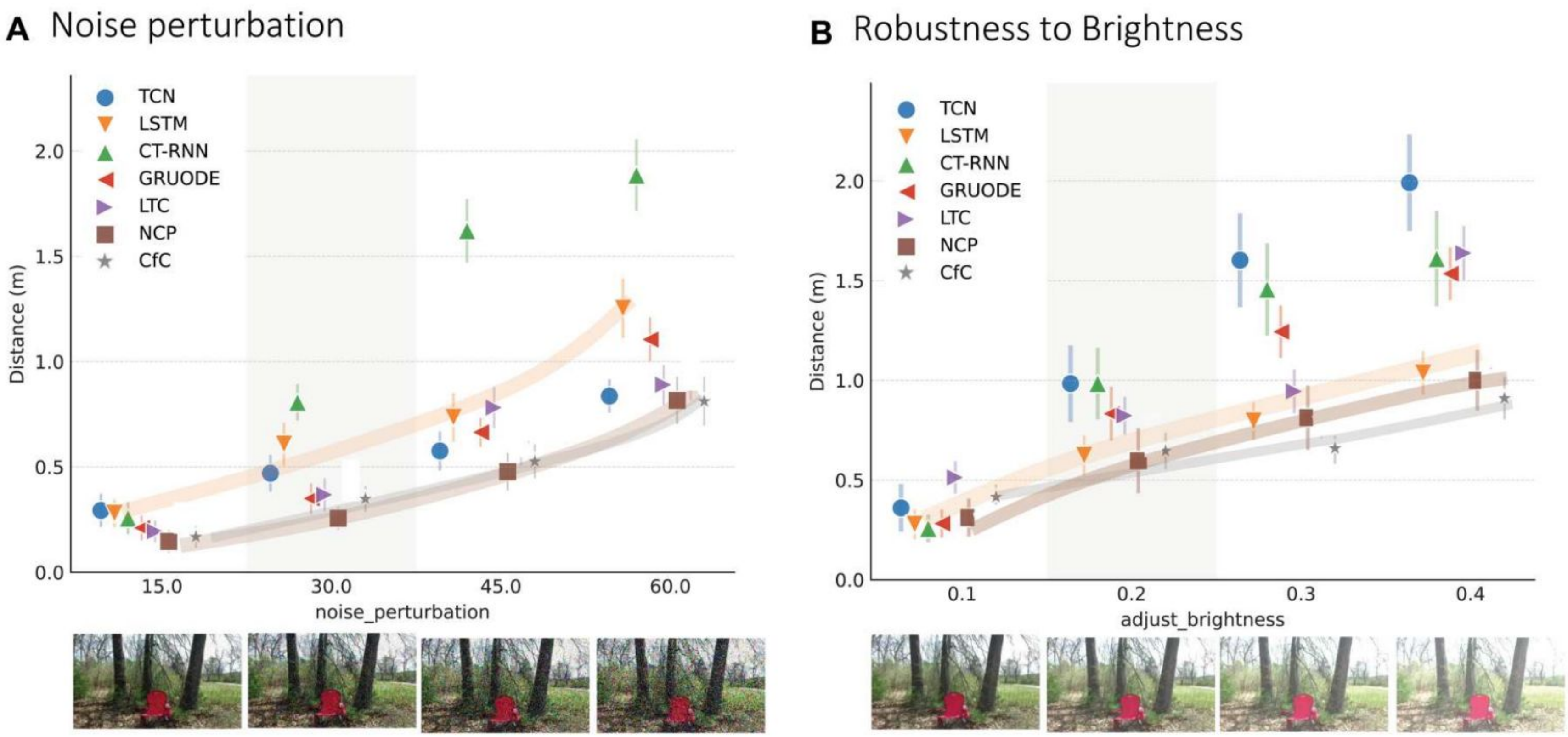
Algorithm	Environment			
	Training Woods	Alt. Woods	Urban Lawn	Urban Patio
LSTM	82.5%	92.5%	62.5%	27.5%
GRU-ODE	50%	17.5%	32.5%	17.5%
ODE-RNN	62.5%	82.5%	17.5%	25%
TCN	7.5%	0%	0%	0%
NCP (ours)	100%	95%	57.5%	52.5%
CfC (ours)	85%	95%	90%	67.5%

The Evolution of LNNs: From Worms to ODEs

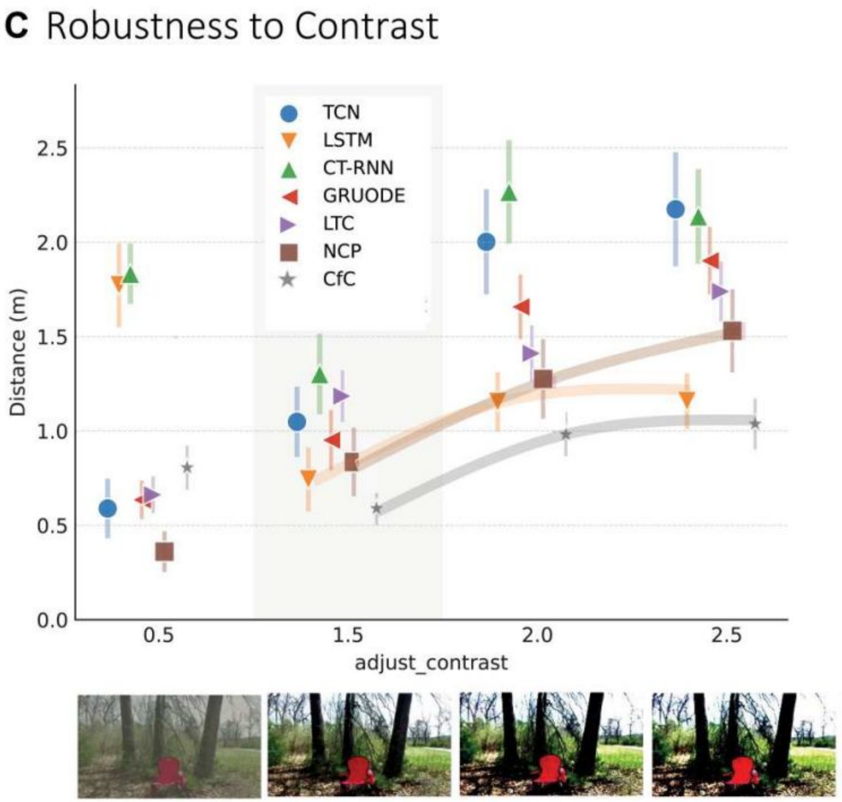
Table 2. Range test evaluation of the trained policies in the Training Woods environment. Quantitative results are success rates. Higher is better. (N = 10)

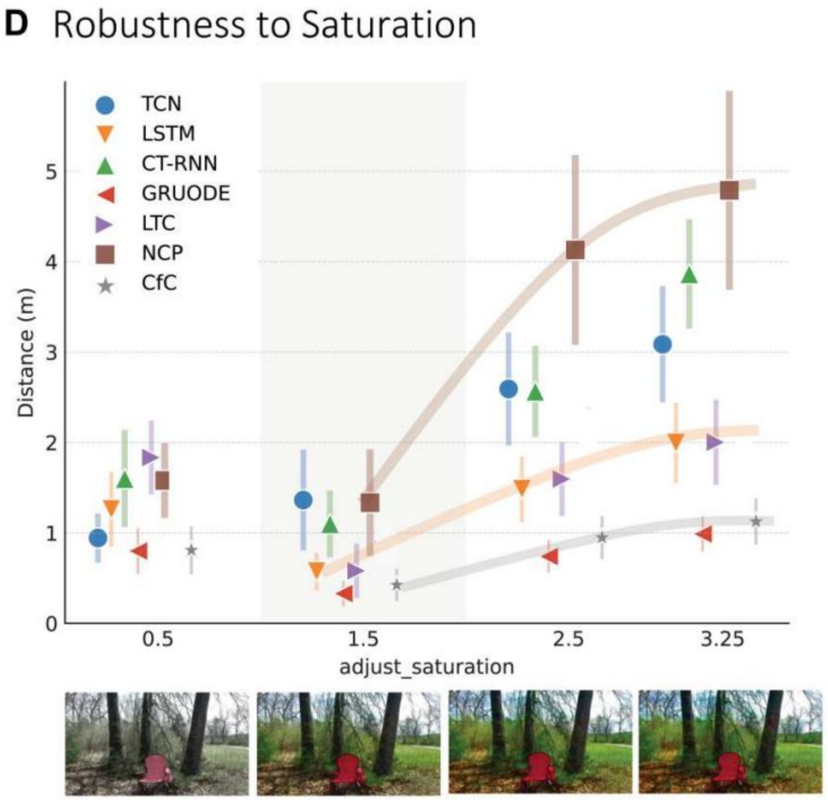
Algorithm	**Initial distance (m)**		
	10	**20**	**30**
LSTM	100%	0%	0%
GRU-ODE	70%	0%	0%
ODE-RNN	80%	10%	0%
TCN	5%	0%	0%
NCP (ours)	100%	50%	10%
CfC (ours)	100%	90%	20%

The Evolution of LNNs: From Worms to ODEs

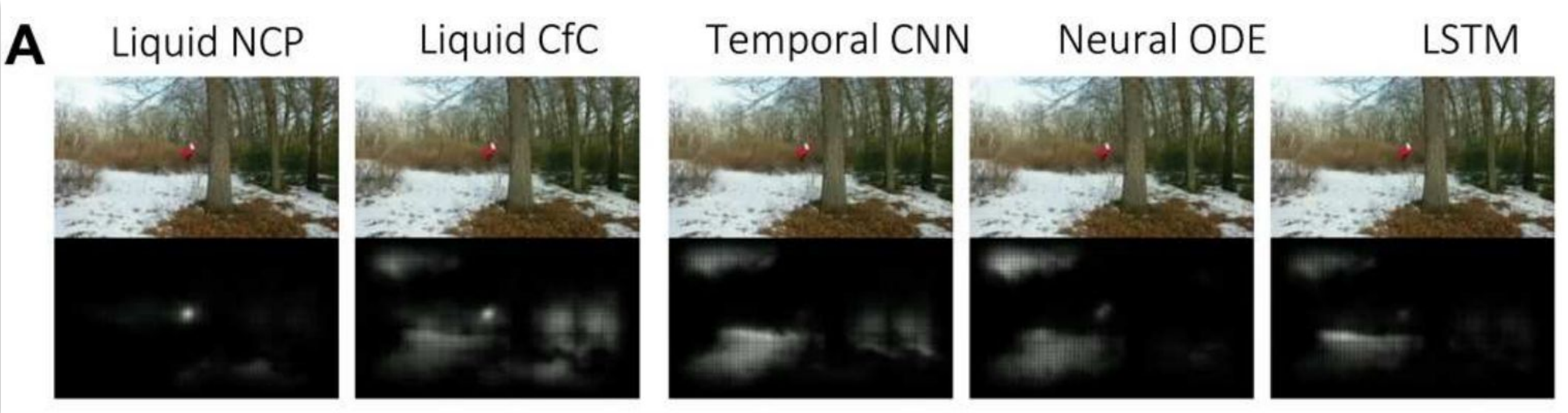


The Evolution of LNNs: From Worms to ODEs





The Evolution of LNNs: From Worms to ODEs

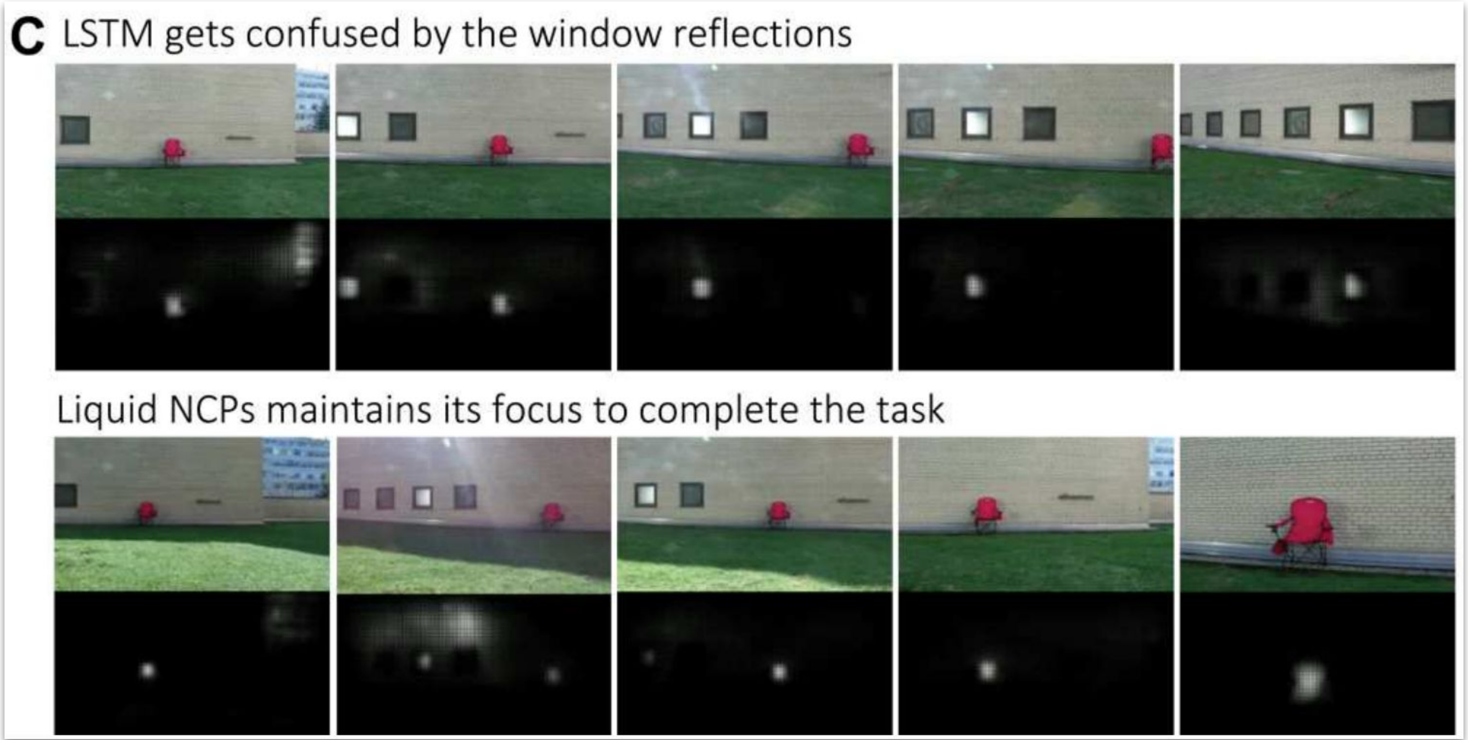


- A frame illustration of the attention profile of different networks computed by the VisualBackProp saliency map computation algorithm. The top row shows the input image frame. The bottom row shows the saliency map. The brighter regions indicate a greater concentration of the convolutional layer’s attention. Saliency scores range between 0 (black) and 1 (white).

The Evolution of LNNs: From Worms to ODEs

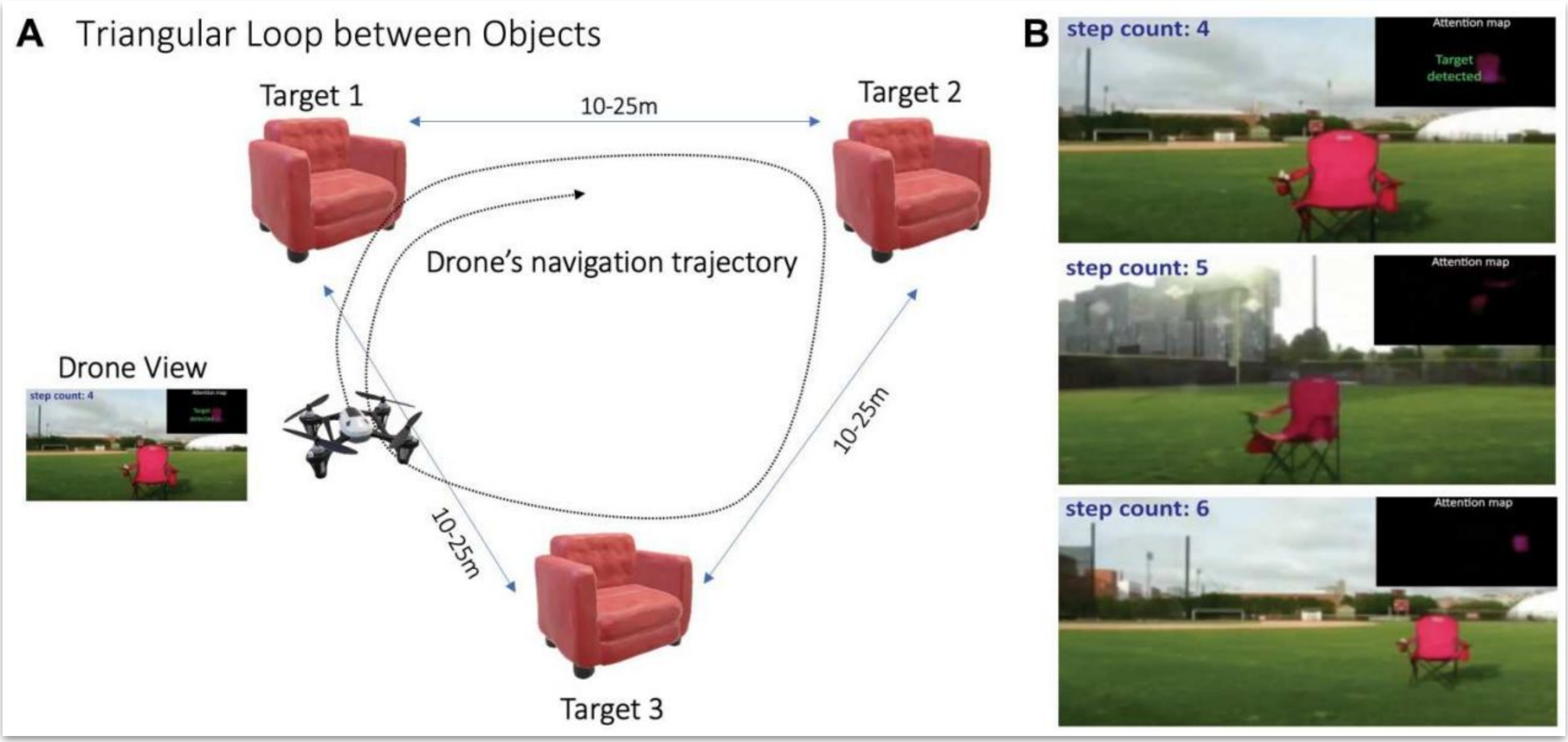
- A flight case study where an LSTM agent loses its target and gets confused by an adversary. Left to right shows the progress time.

The Evolution of LNNs: From Worms to ODEs



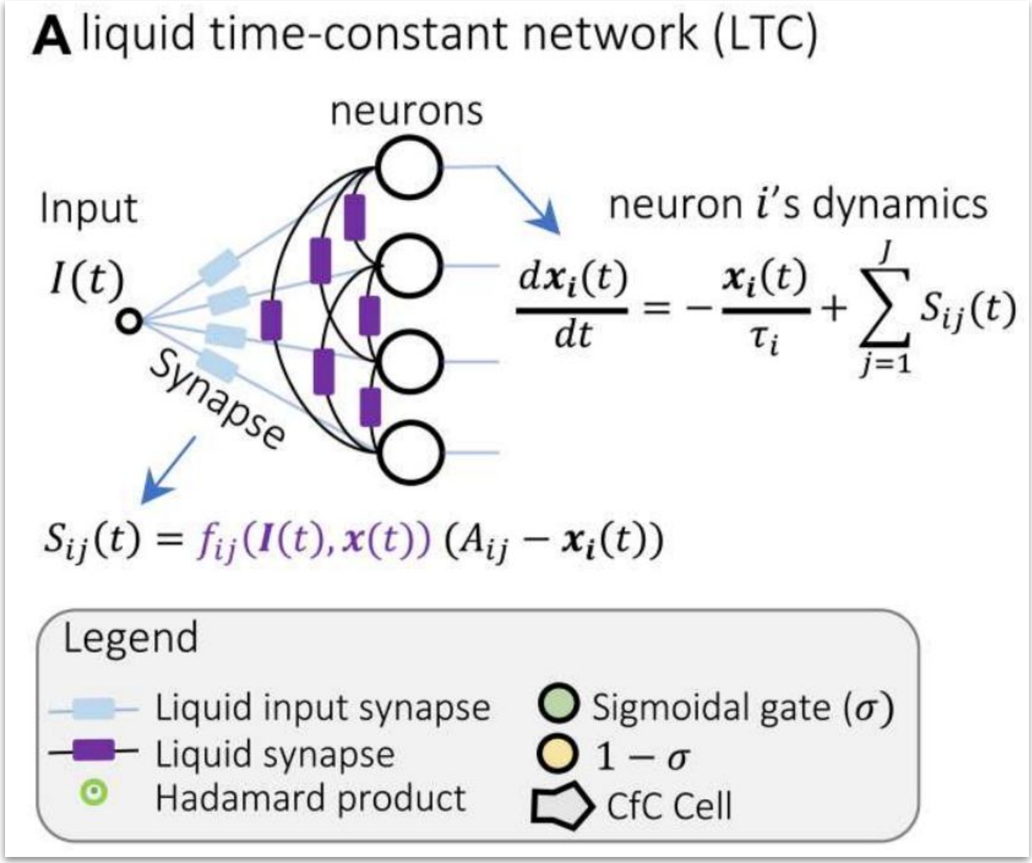
- A similar scenario as in (B), in a different OOD testing environment.

The Evolution of LNNs: From Worms to ODEs



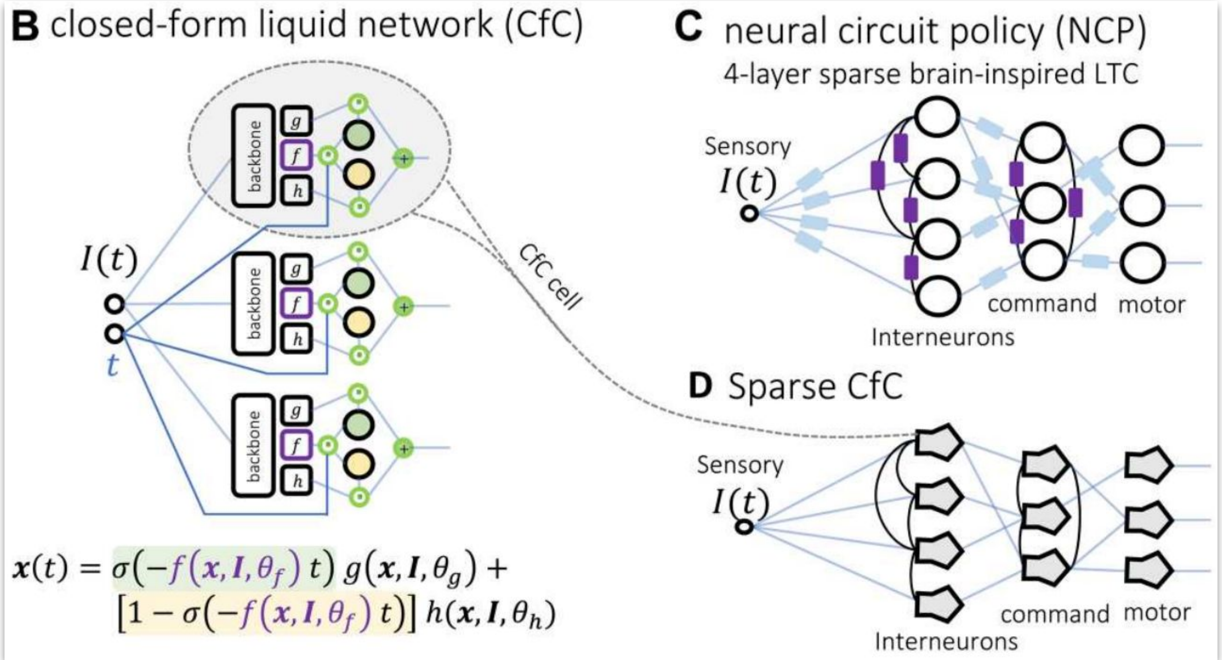
- Triangularloop between objects. (A) The triangular infinite loop task setup. (B) Time instances of drone viewstogether with their attention map atsteps 4, 5, and 6 of the infinite loop

The Evolution of LNNs: From Worms to ODEs



- Liquid neural networks. (A) Schematic demonstration of a fully connected LTC layer
- The dynamics of a single neuron *i* is given, where $x_i\ (t)$ represents the state of the neuron *i* and *τ* represents the neuronal time constant. Synapses are given by a nonlinear function, where *f (.)* represents a sigmoidal nonlinearity and *Ai j* is a reversal potential parameter for each synapse from neurons *j* to *i*.

The Evolution of LNNs: From Worms to ODEs



$$x(t) = \sigma(-f(x, I, \theta_f)\, t)\, g(x, I, \theta_g) + [1 - \sigma(-f(x, I, \theta_f)\, t)]\, h(x, I, \theta_h)$$

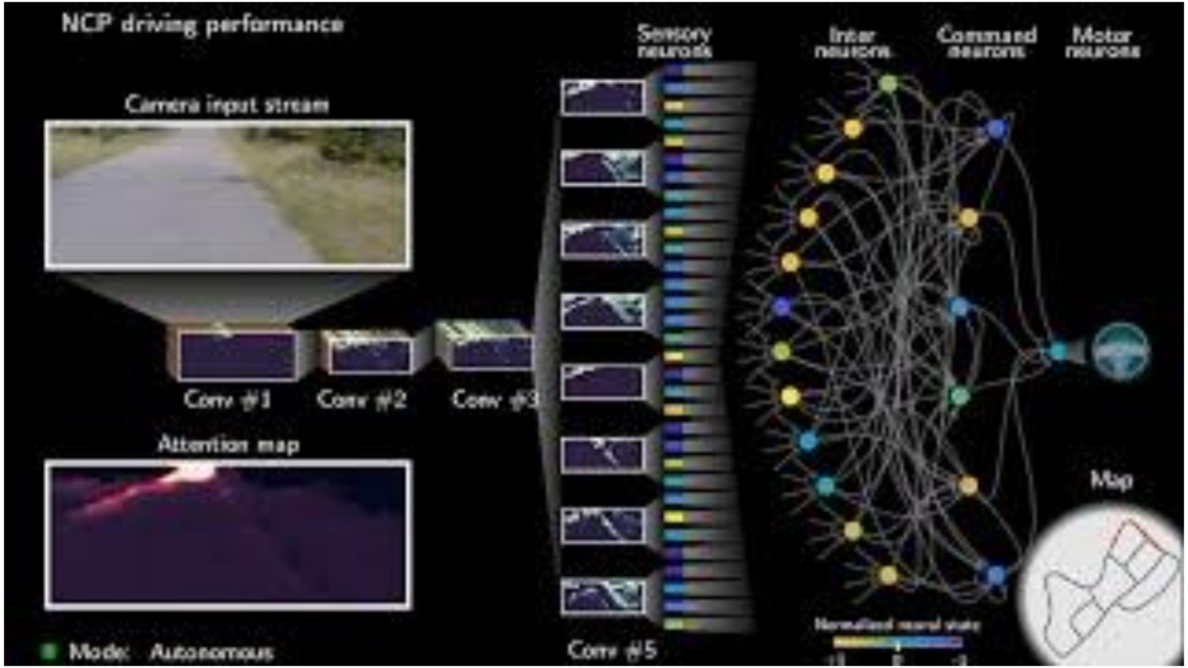
- (B) Schematic representation of a closed-form liquid network (CfC) together with its state equation. (C) A schematic representation of an NCP. (D) CfC. A sparse neural circuit with the same architectural motifs as NCPs, this time with CfC neural dynamics.

The Evolution of LNNs: From Worms to ODEs

- Drones navigate unseen environments with liquid neural networks

Sources: https://www.youtube.com/watch?v=RPFbk6D_dNw&t=48s

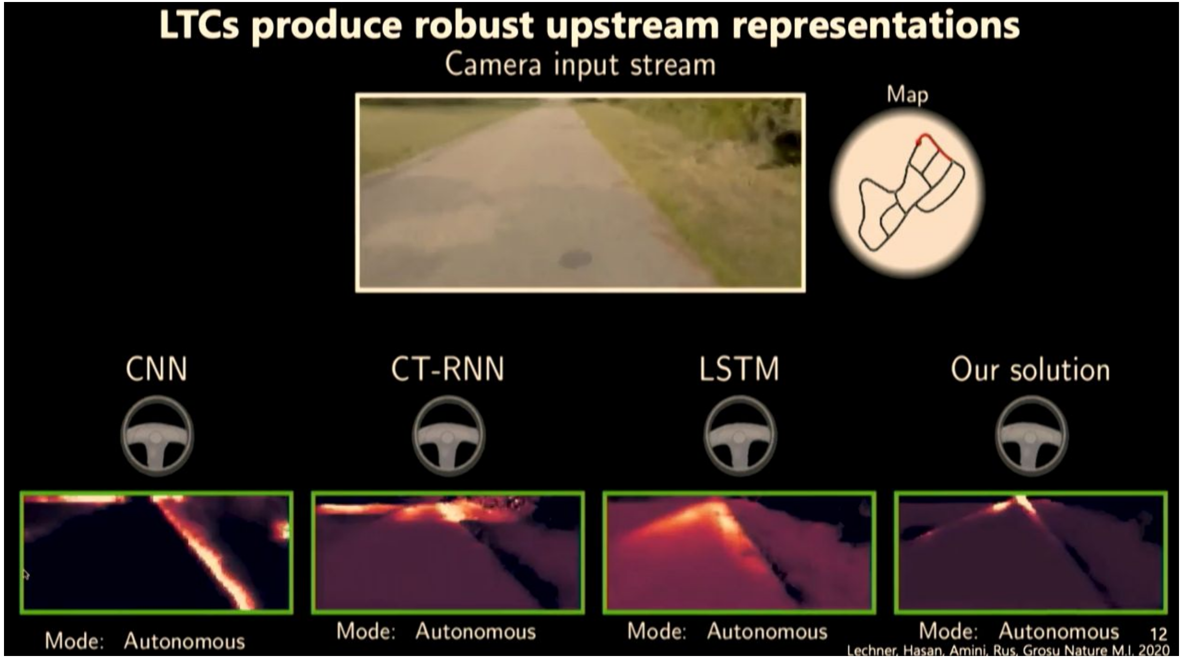
The Evolution of LNNs: From Worms to ODEs



- We have to consider many factors
- Car driving mechanics
- Road definition
- Artificial sensors
- Collision detection
- Traffic simulation
- Live stream variant
- Modeling
- Fine-tuning etc.

Sources: https://www.youtube.com/watch?v=8KBOf7NJh4Y

The Evolution of LNNs: From Worms to ODEs



- Check attention maps
- For a system to work in a real word it must be causal

Sources: https://www.youtube.com/watch?v=0FNkrjVIcuk

The Evolution of LNNs: From Worms to ODEs

Sources: https://www.youtube.com/watch?v=0FNkrjVIcuk

- ဒီ talk မှာက CNN ရဲ့ attention နဲ့ LNNs ရဲ့ attention မှာ ဘယ်လို ကွာတယ်ဆိုတာကို မြင်ရလိမ့်မယ်
- ပြီးတော့ autonomous flight simulation, Cheetah simulation တွေနဲ့ ပတ်သက်တာပါတယ်
- Proposal နဲ့ ပတ်သက်တာလည်း ရှင်းပြထားတယ်

The Evolution of LNNs: From Worms to ODEs

arXiv > cs > arXiv:2504.02352

Help

Computer Science > Information Theory

[Submitted on 3 Apr 2025]

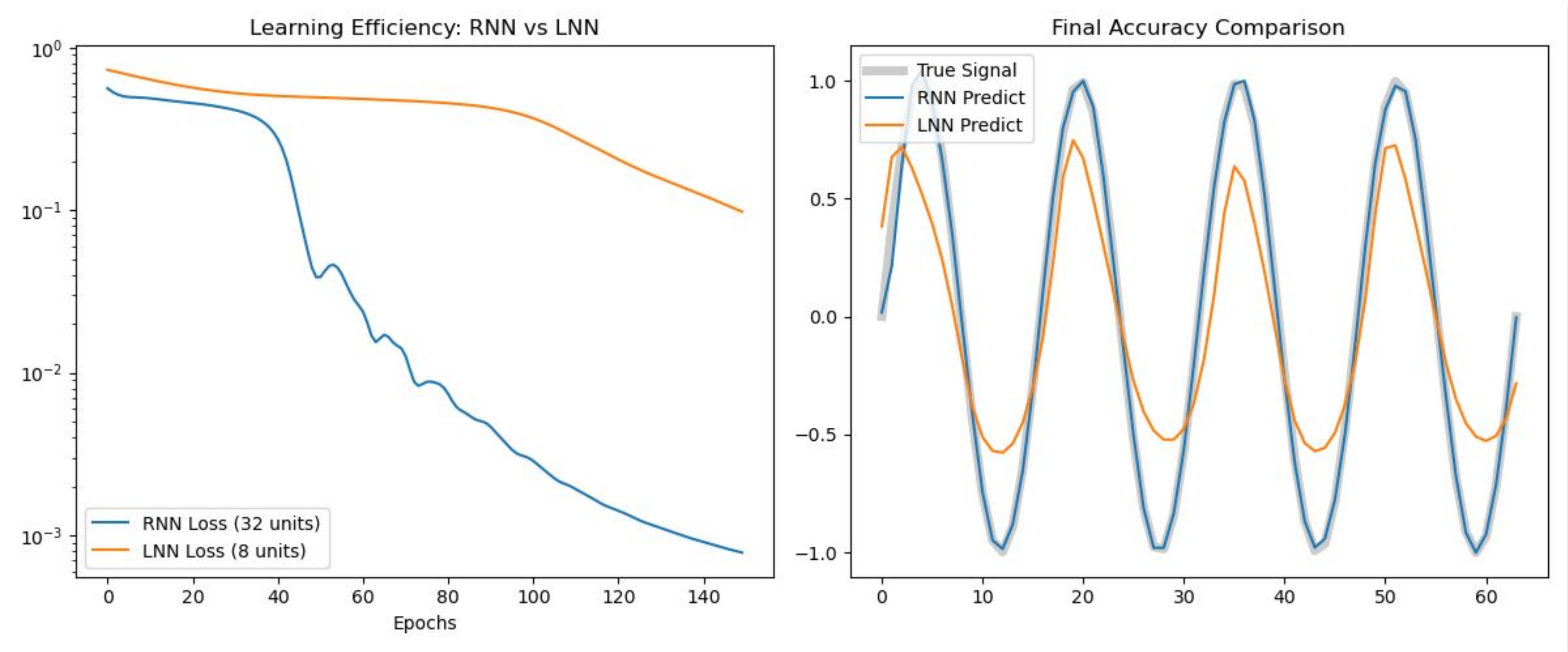
Liquid Neural Networks: Next-Generation AI for Telecom from First Principles

REQUEST CODE

Fenghao Zhu, Xinquan Wang, Chen Zhu, Chongwen Huang

Artificial intelligence (AI) has emerged as a transformative technology with immense potential to reshape the next-generation of wireless networks. By leveraging advanced algorithms and machine learning techniques, AI offers unprecedented capabilities in optimizing network performance, enhancing data processing efficiency, and enabling smarter decision-making processes. However, existing AI solutions face significant challenges in terms of robustness and interpretability. Specifically, current AI models exhibit substantial performance degradation in dynamic environments with varying data distributions, and the black-box nature of these algorithms raises concerns regarding safety, transparency, and fairness. This presents a major challenge in integrating AI into practical communication systems. Recently, a novel type of neural network, known as the liquid neural networks (LNNs), has been designed from first principles to address these issues. In this paper, we explore the potential of LNNs in telecommunications. First, we illustrate the mechanisms of LNNs and highlight their unique advantages over traditional networks. Then we unveil the opportunities that LNNs bring to future wireless networks. Furthermore, we discuss the challenges and design directions for the implementation of LNNs. Finally, we summarize the performance of LNNs in two case studies.

LSTM vs LNNs



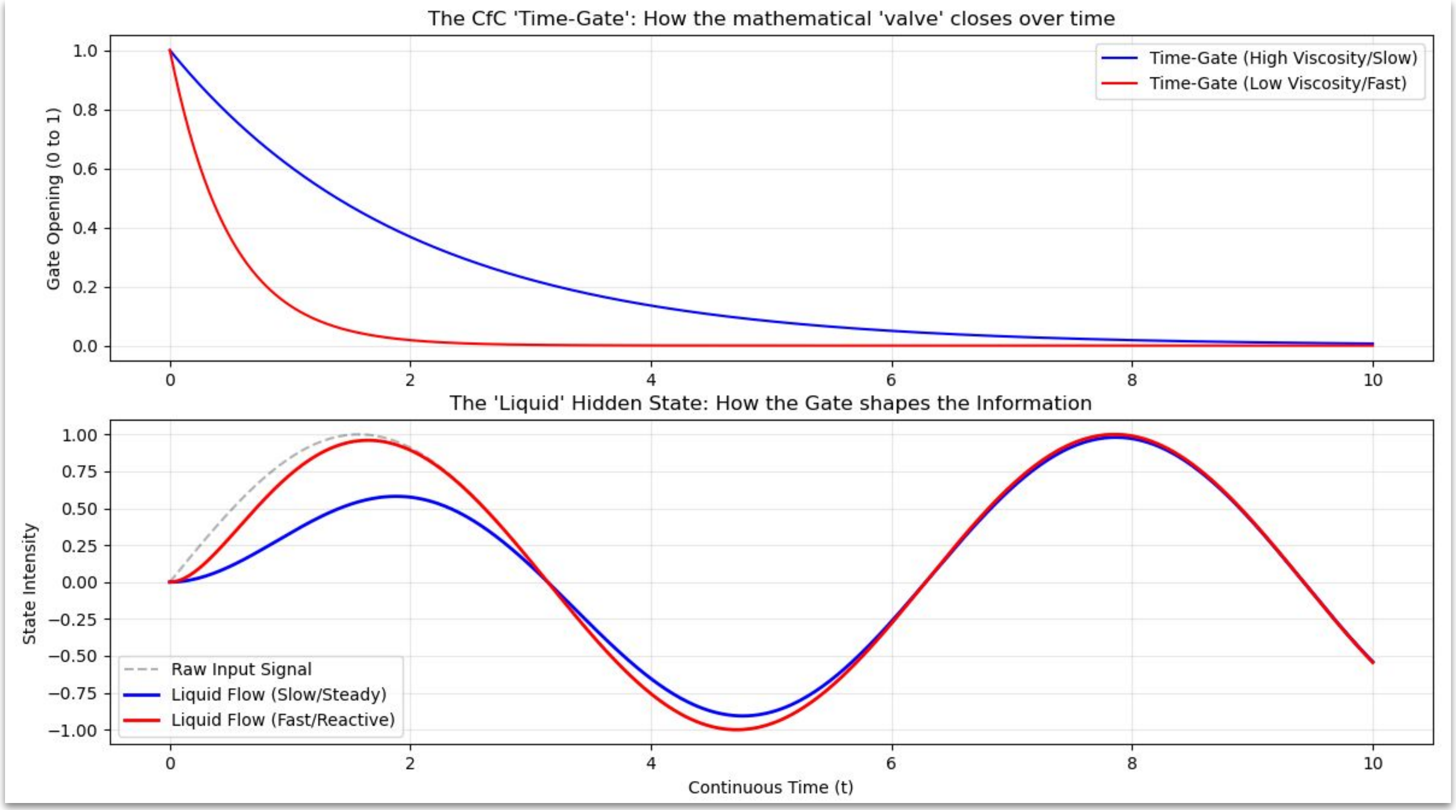
Reference link: https://www.kaggle.com/code/newtonbaba12345/liquid-neural-networks

Closed-form Continuous-time (CfC)

```python
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def visualize_cfc_gates():
    t = np.linspace(0, 10, 500)
    
    # Simulate a signal (Information)
    input_signal = np.sin(t)
    
    # 1. The Time-Gate: exp(-t * tau)
    # This represents the "Viscosity" of the liquid.
    # Higher tau = faster decay/adaptation.
    tau_slow = 0.5
    tau_fast = 2.0
    
    time_gate_slow = np.exp(-t * tau_slow)
    time_gate_fast = np.exp(-t * tau_fast)
    
    # 2. The Gating Mechanism: h(t) = G*gate + I*(1-gate)
    # This is the "Closed-form" solution logic.
    liquid_flow_slow = input_signal * (1 - time_gate_slow)
    liquid_flow_fast = input_signal * (1 - time_gate_fast)
```

- Visualization of CfC gates

Closed-form Continuous-time (CfC)



Closed-form Continuous-time (CfC)

```python
import tensorflow as tf
from ncps.tf import LTC, CfC
from ncps import wirings
import numpy as np
import time
import matplotlib.pyplot as plt

# 1. Setup Data
N = 100
t = np.linspace(0, 10, N)
x = np.sin(t).reshape(1, N, 1).astype(np.float32)
y = np.cos(t).reshape(1, N, 1).astype(np.float32)

# 2. Build LTC (The "Old" Liquid)
wiring = wirings.FullyConnected(16, 1)
ltc_model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(input_shape=(None, 1)),
    LTC(wiring, return_sequences=True)
])
ltc_model.compile(optimizer='adam', loss='mse')

```

- Python library Neural Circuit Policies (NCPs)
- Speed comparison code for Liquid Time-Constants (LTC) and CfC

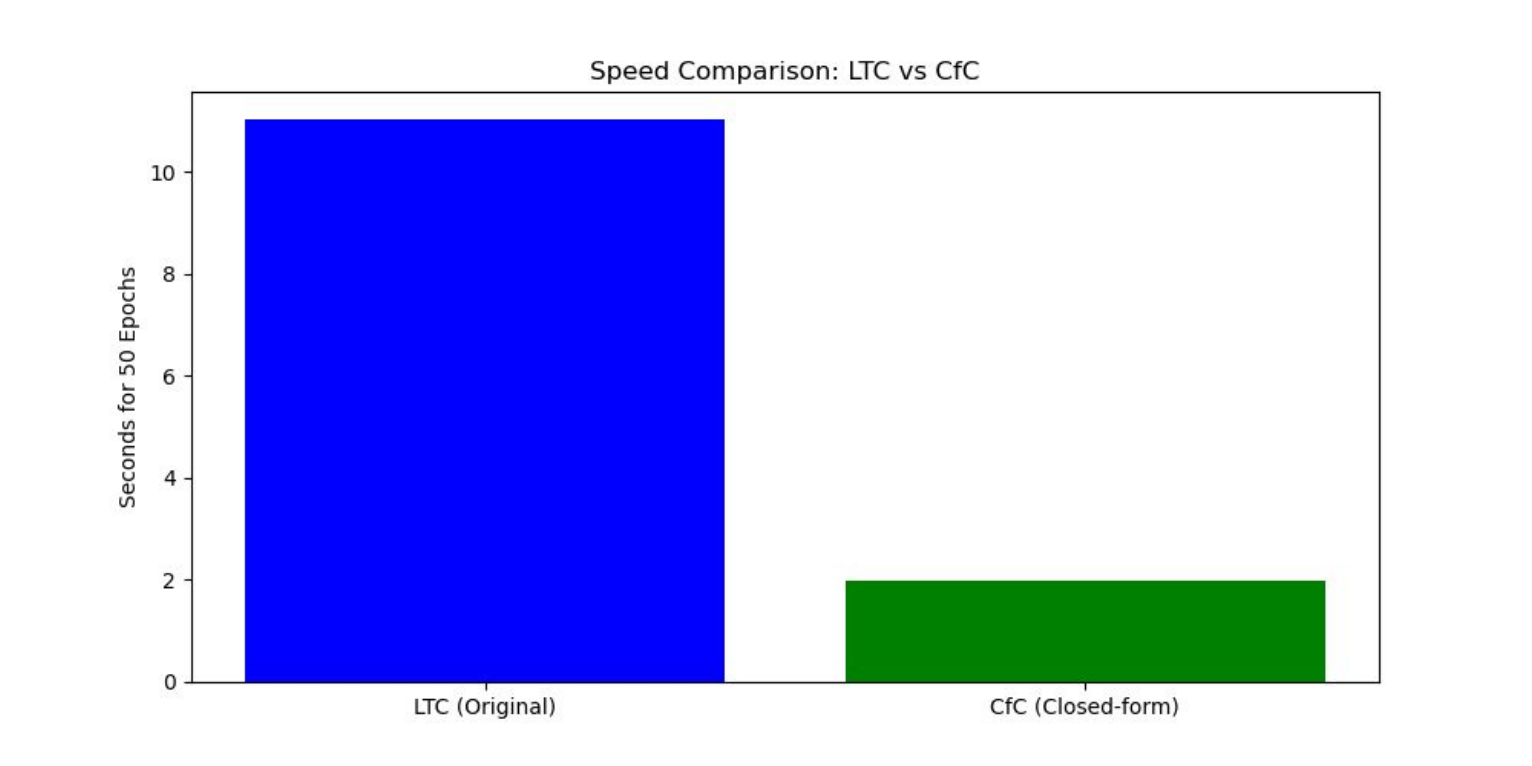
Closed-form Continuous-time (CfC)

```
# 3. Build CfC (The "New" Liquid)
cfc_model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(input_shape=(None, 1)),
    # CfC is a drop-in layer. No complex wiring required by default.
    CfC(16, return_sequences=True)
])
cfc_model.compile(optimizer='adam', loss='mse')

# 4. Benchmarking Speed
start = time.time()
ltc_model.fit(x, y, epochs=50, verbose=0)
ltc_time = time.time() - start

start = time.time()
cfc_model.fit(x, y, epochs=50, verbose=0)
cfc_time = time.time() - start

print(f"LTC (ODE-based) Training Time: {ltc_time:.2f}s")
print(f"CfC (Closed-form) Training Time: {cfc_time:.2f}s")
```

Closed-form Continuous-time (CfC)

```
# 5. Visualizing Architecture
# Note: CfC can be 'wired', but by default it acts as a high-speed continuous layer.
plt.figure(figsize=(10, 5))
plt.bar(['LTC (Original)', 'CfC (Closed-form)'], [ltc_time, cfc_time], color=['blue',
'green'])
plt.ylabel("Seconds for 50 Epochs")
plt.title("Speed Comparison: LTC vs CfC")
plt.show()
```

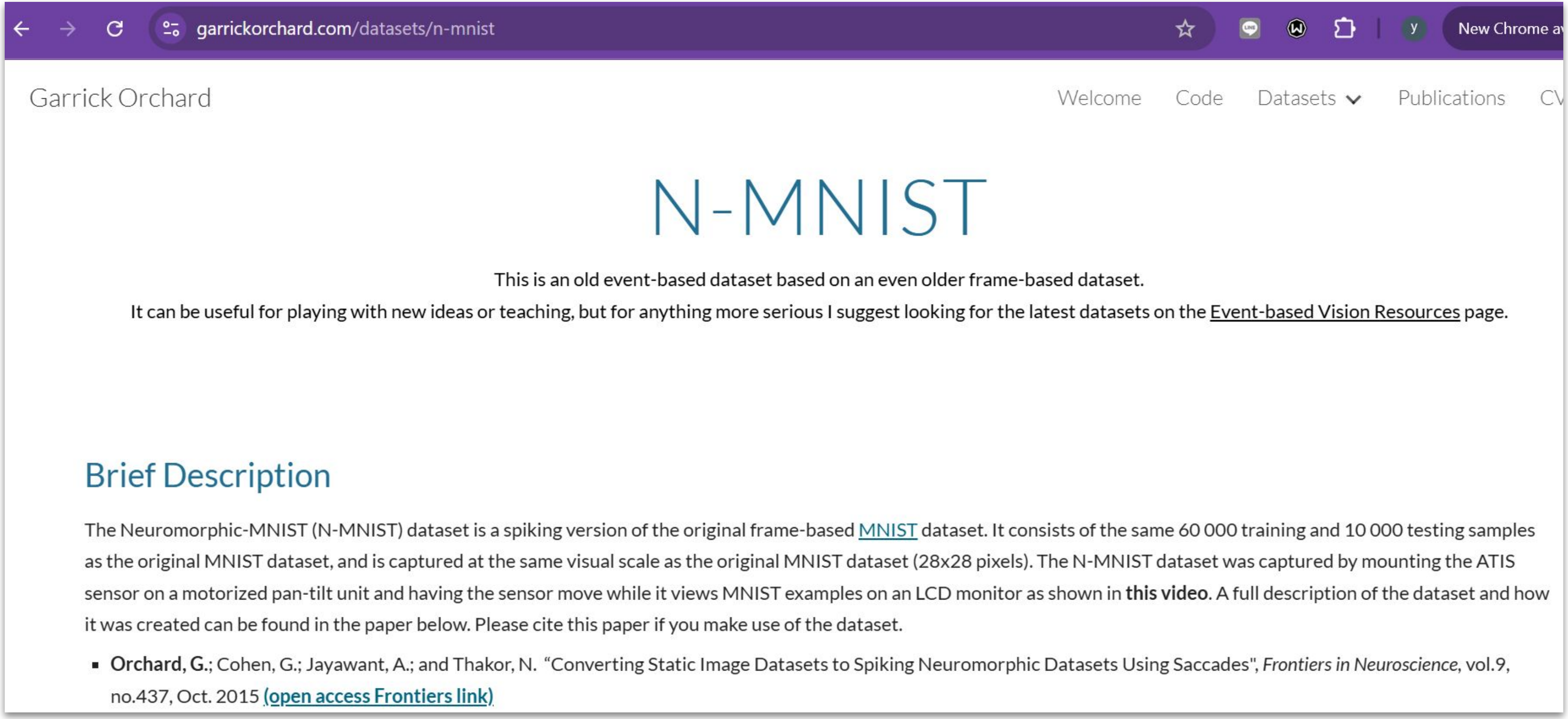
Closed-form Continuous-time (CfC)



Sequential Pattern Recognition with LNNs

- We conduct a large-scale benchmarking study across three distinct sequential modalities:
  1. Neuromorphic event-based data (N-MNIST)
  2. Stroke-based drawing coordinates (Google QuickDraw)
  3. Visual handwriting recognition (IAM Dataset)

Sequential Pattern Recognition with LNNs



garrickorchard.com/datasets/n-mnist

Garrick Orchard

Welcome Code Datasets Publications CV

N-MNIST

This is an old event-based dataset based on an even older frame-based dataset.

It can be useful for playing with new ideas or teaching, but for anything more serious I suggest looking for the latest datasets on the Event-based Vision Resources page.

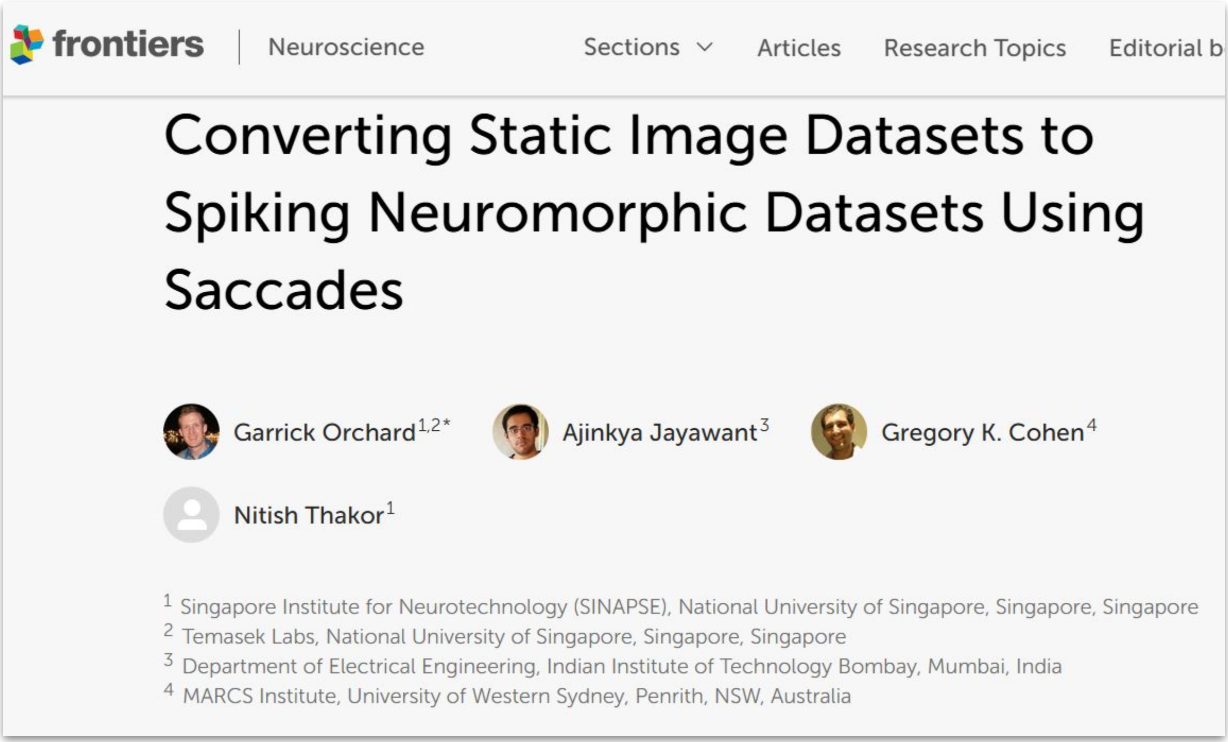
Brief Description

The Neuromorphic-MNIST (N-MNIST) dataset is a spiking version of the original frame-based MNIST dataset. It consists of the same 60 000 training and 10 000 testing samples as the original MNIST dataset, and is captured at the same visual scale as the original MNIST dataset (28x28 pixels). The N-MNIST dataset was captured by mounting the ATIS sensor on a motorized pan-tilt unit and having the sensor move while it views MNIST examples on an LCD monitor as shown in **this video**. A full description of the dataset and how it was created can be found in the paper below. Please cite this paper if you make use of the dataset.

- **Orchard, G.**; Cohen, G.; Jayawant, A.; and Thakor, N. "Converting Static Image Datasets to Spiking Neuromorphic Datasets Using Saccades", *Frontiers in Neuroscience*, vol.9, no.437, Oct. 2015 (open access Frontiers link)

Source: https://www.garrickorchard.com/datasets/n-mnist

Sequential Pattern Recognition with LNNs



frontiers | Neuroscience   Sections   Articles   Research Topics   Editorial b

Converting Static Image Datasets to Spiking Neuromorphic Datasets Using Saccades

Garrick Orchard[1,2*]   Ajinkya Jayawant[3]   Gregory K. Cohen[4]

Nitish Thakor[1]

[1] Singapore Institute for Neurotechnology (SINAPSE), National University of Singapore, Singapore, Singapore
[2] Temasek Labs, National University of Singapore, Singapore, Singapore
[3] Department of Electrical Engineering, Indian Institute of Technology Bombay, Mumbai, India
[4] MARCS Institute, University of Western Sydney, Penrith, NSW, Australia

- 2015 လောက်က စာတမ်းပါ
- နာမည်ကြီး Static ပုံဒေတာတွေ ဖြစ်တဲ့ MNIST နဲ့ Caltech101ကို neuromorphic vision dataset အဖြစ် ပြောင်းတဲ့ စာတမ်း

Source: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2015.00437/full

Sequential Pattern Recognition with LNNs

FIGURE 2 | (A) A picture of the ATIS mounted on the pan tilt unit used in the conversion system. **(B)** The ATIS placed viewing the LCD monitor.

Source: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2015.00437/full

Sequential Pattern Recognition with LNNs

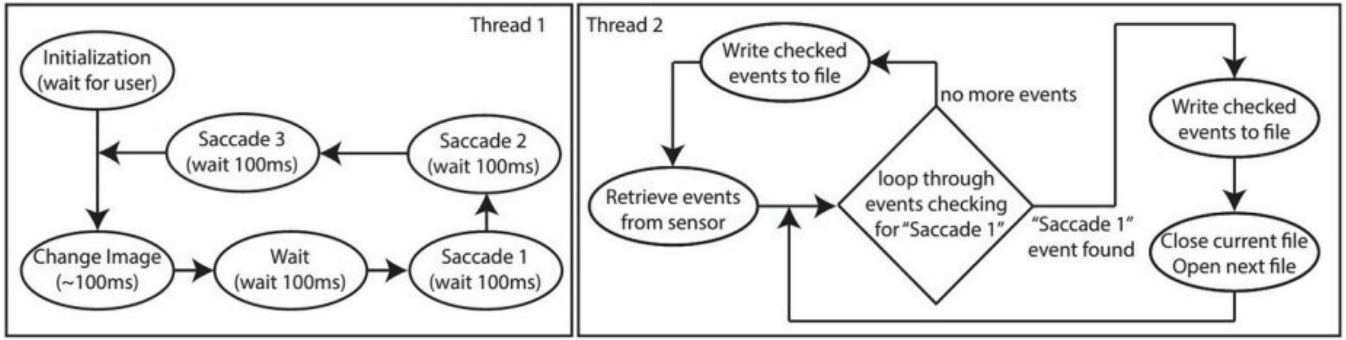
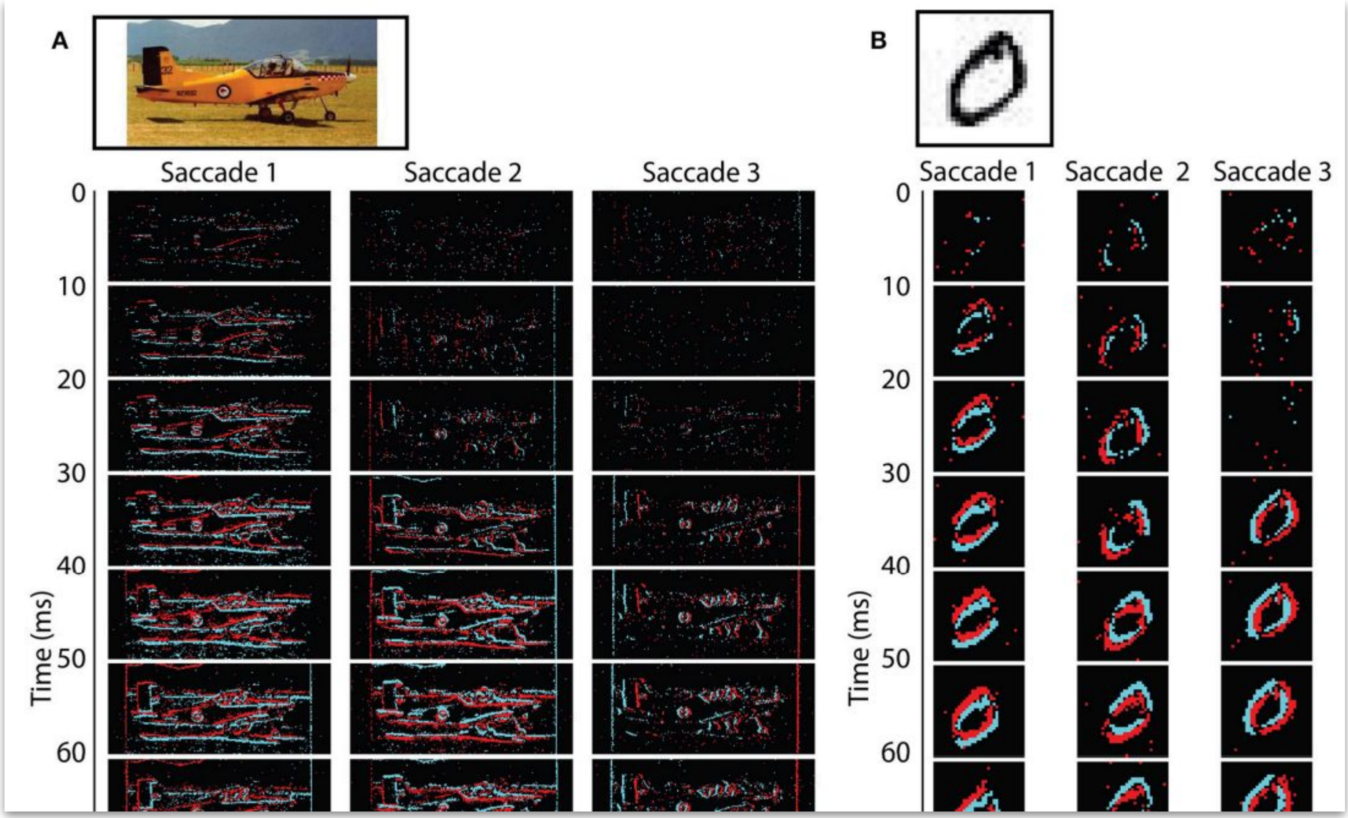


FIGURE 3 | Software flow diagram of the PC code showing two threads. Thread 1 **(left)** controls the timing of microsaccades and changing of images on the screen, while thread 2 **(right)** controls acquisition of the event data and writing it to disk.

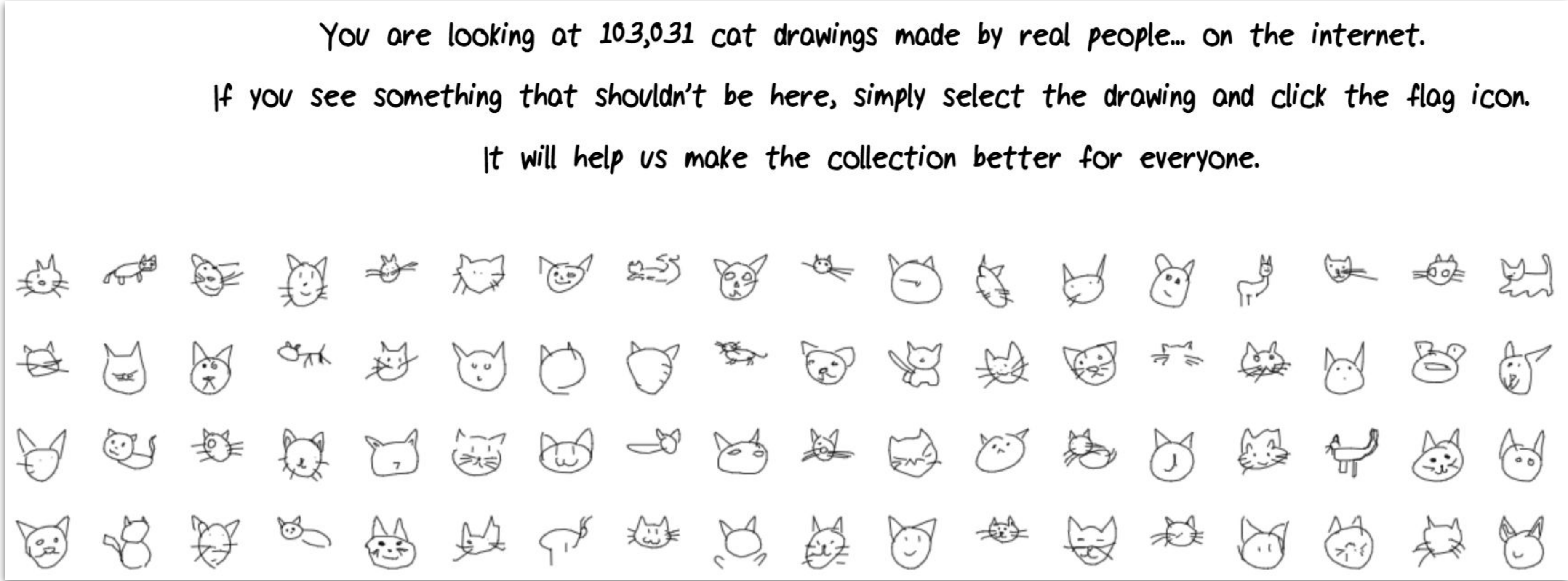
Source: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2015.00437/full

Sequential Pattern Recognition with LNNs



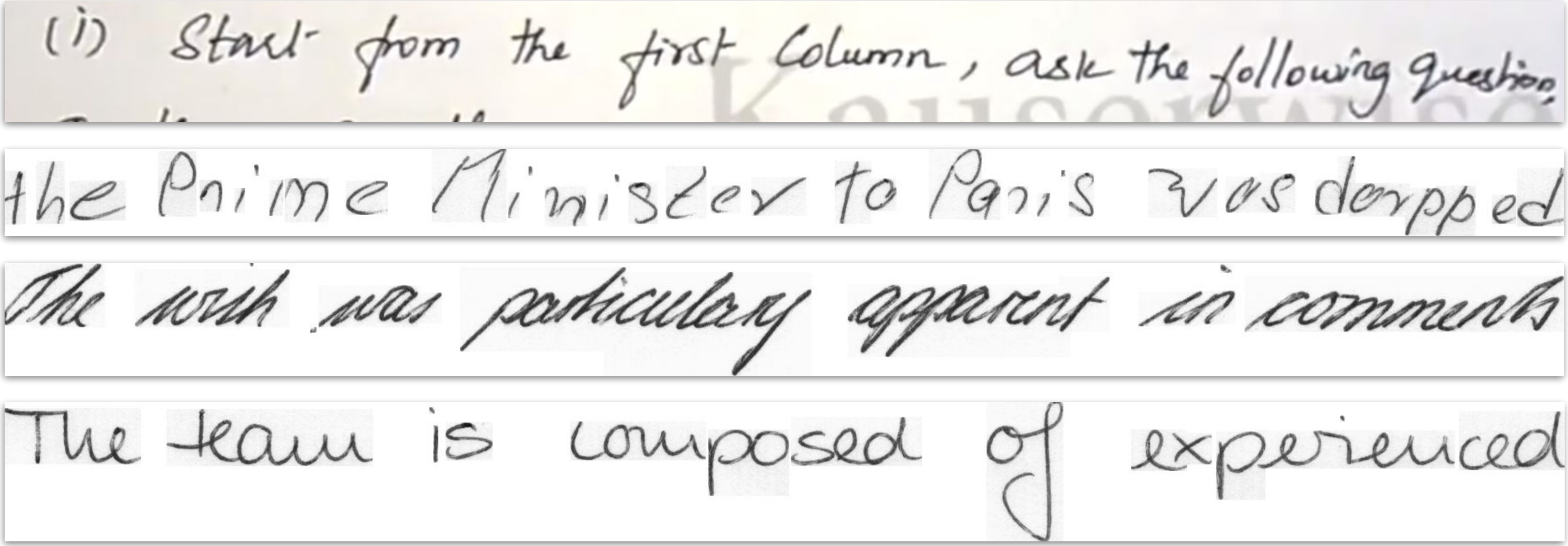
Sequential Pattern Recognition with LNNs

Sequential Pattern Recognition with LNNs



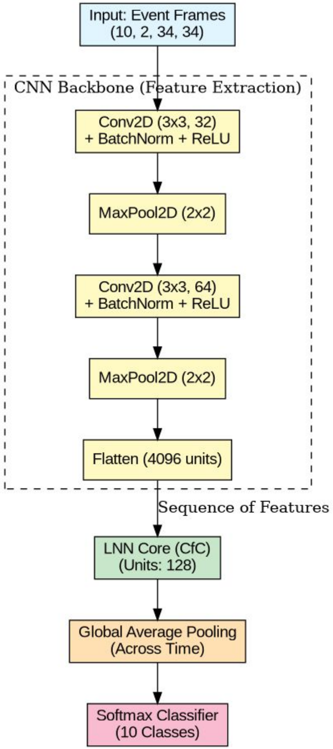
Source: https://quickdraw.withgoogle.com/data

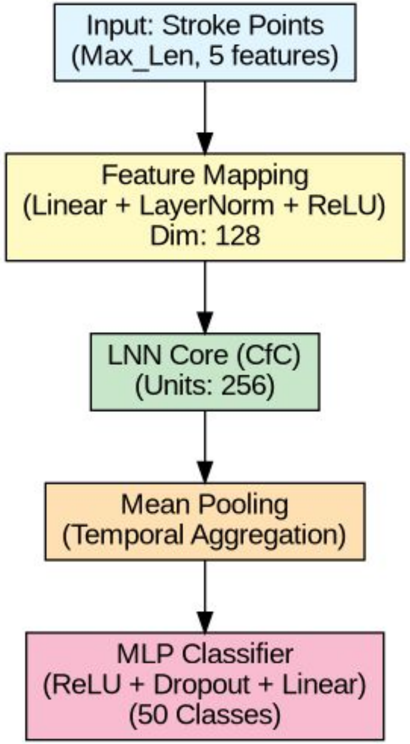
Sequential Pattern Recognition with LNNs



Source: https://github.com/tuandoan998/Handwritten-Text-Recognition

Sequential Pattern Recognition with LNNs





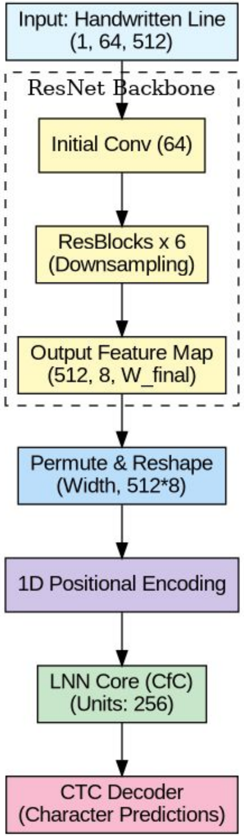


Fig. Architecture for N-MNIST, QuickDraw and IAM

Sequential Pattern Recognition with LNNs

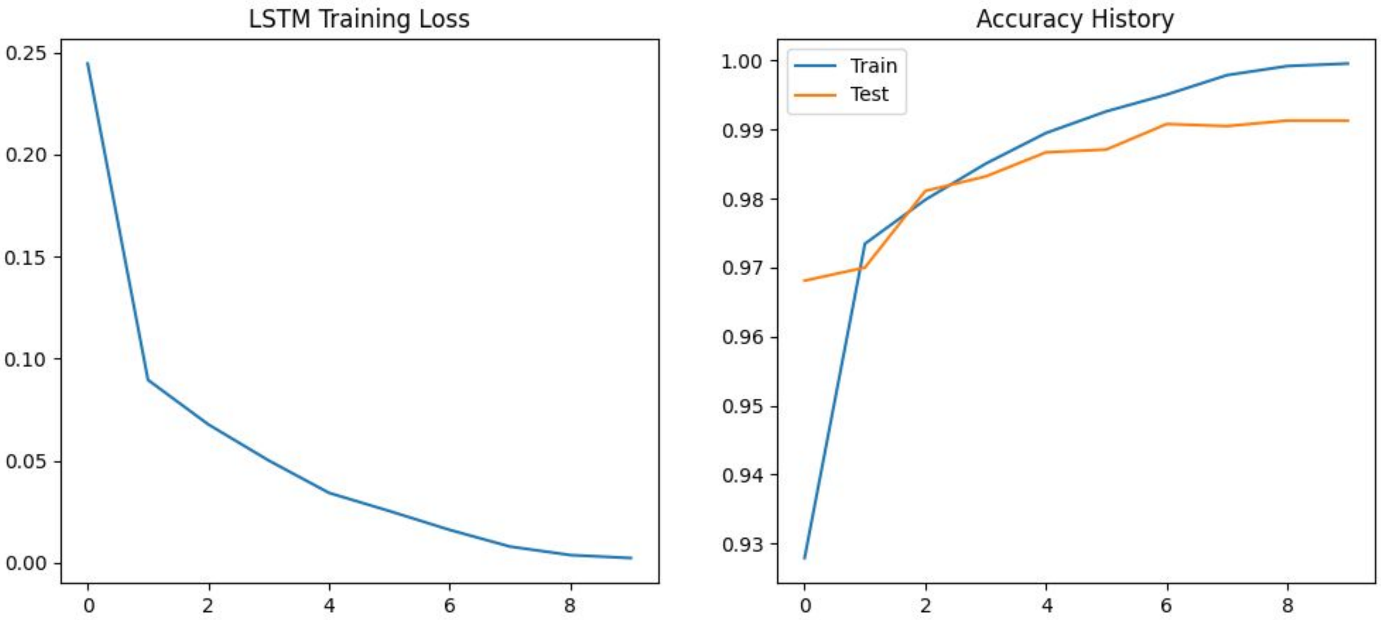


Fig. N-MNIST LSTM training loss and accuracy graph

Sequential Pattern Recognition with LNNs

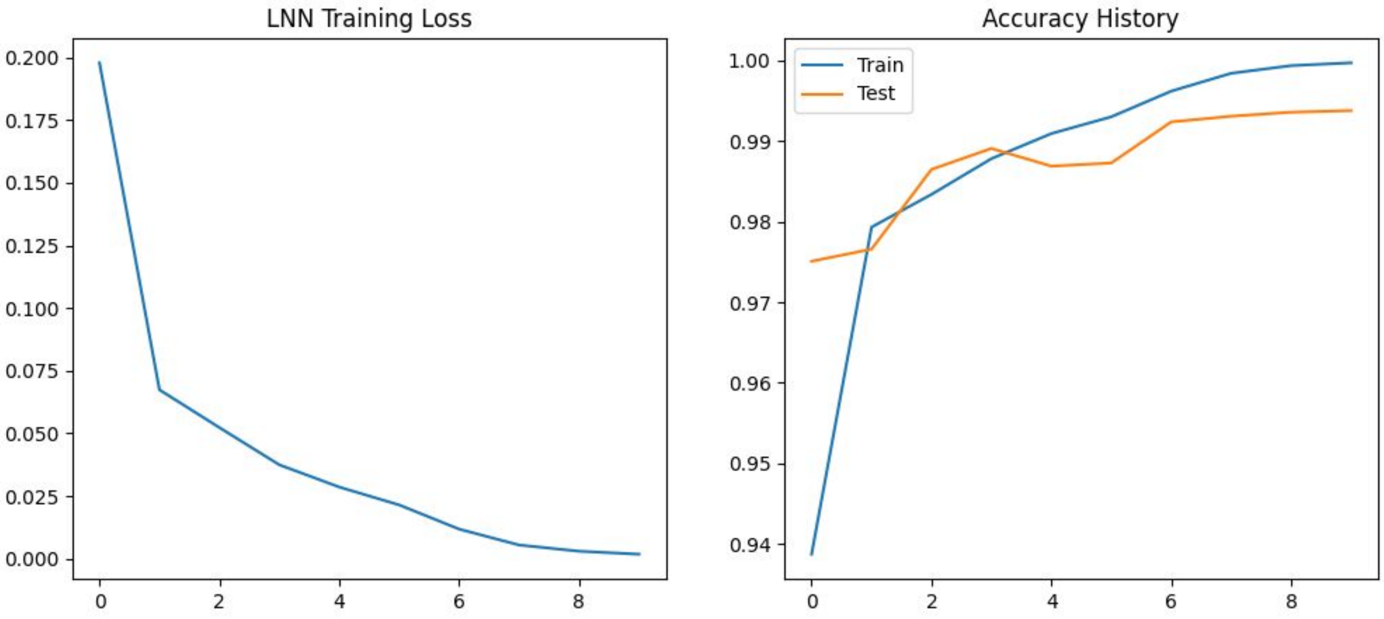


Fig. N-MNIST LNNs training loss and accuracy graph

Sequential Pattern Recognition with LNNs

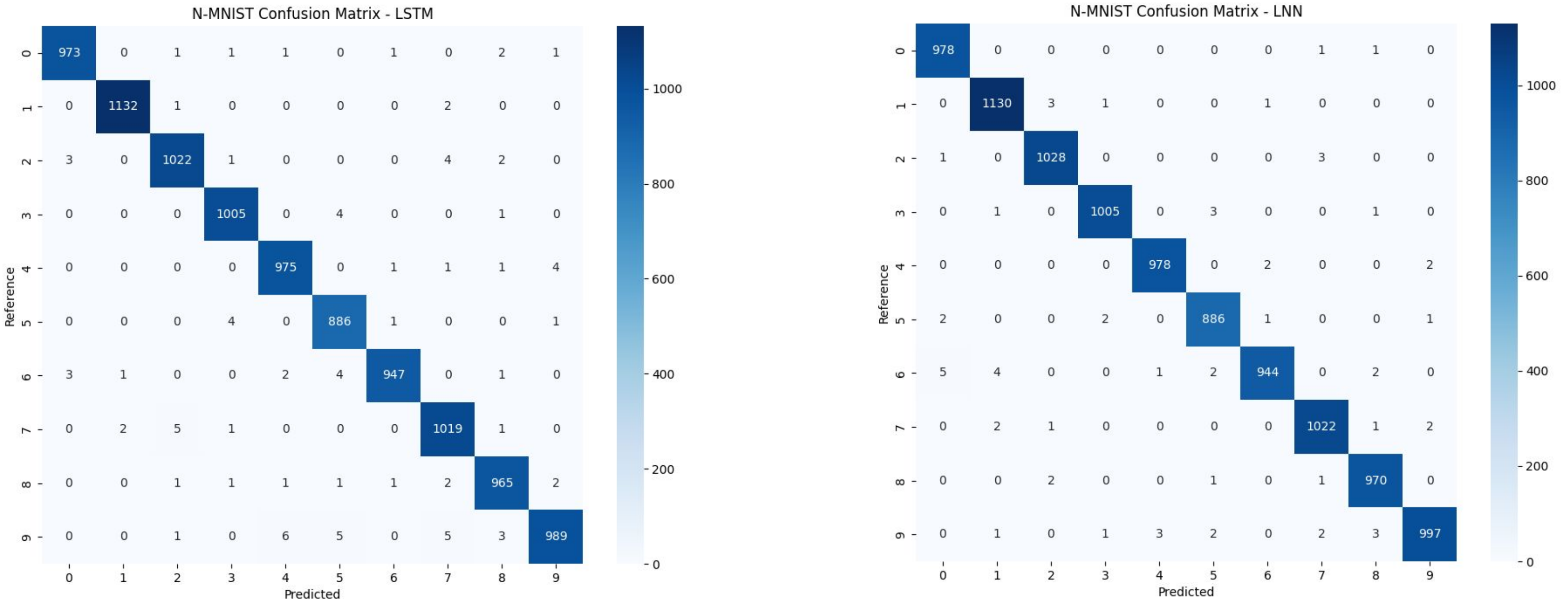


Fig. Confusion matrix for N-MNIST dataset, Left: LSTM, Right: LNNs

Sequential Pattern Recognition with LNNs

Dataset	Model	Metric	Train	Test
N-MNIST	**LNN**	Acc	0.9997	**0.9938**
	LSTM	Acc	0.9996	0.9913
QuickDraw	LNN	Acc	0.9978	0.9577
	LSTM	Acc	1.0000	**0.9701**
IAM	LNN	CER	0.1717	0.1237
	LSTM	CER	0.0784	**0.1090**

Table. Quantitative Benchmark: LNNs vs. LSTM

Fine-tuning with Liquid Foundation Model

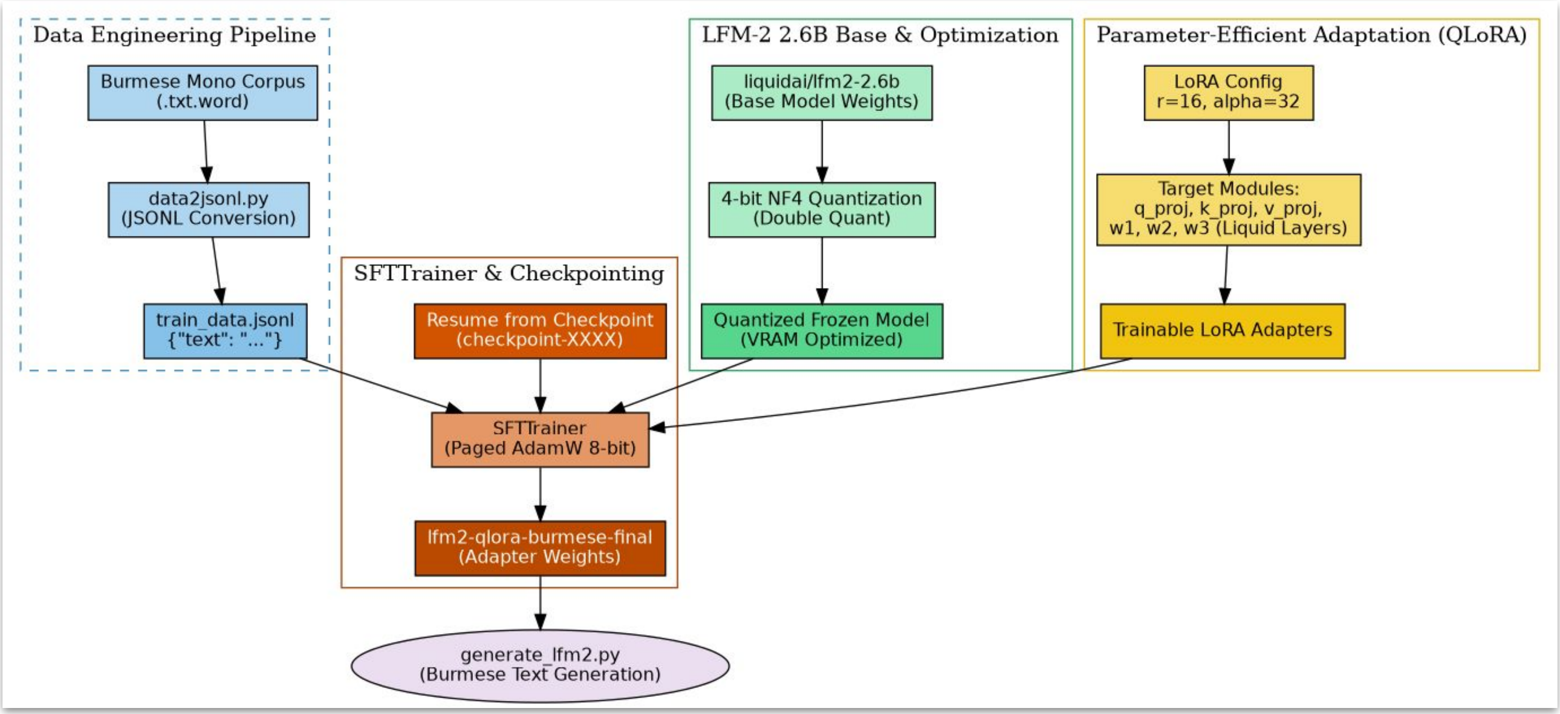


Fig. Overview of Supervised Fine-Tuning with LFM-2-2 6B Base model

Fine-tuning with Liquid Foundation Model

ye@lst-hpc3090: ~/exp/myMono_LFM

```
{'loss': 0.5701, 'grad_norm': 2.3793764114379883, 'learning_rate': 1.3634194559956373e-08, 'entropy': 0.6031446309760213, 'num_tokens':
 83132603.0, 'mean_token_accuracy': 0.8193988040089607, 'epoch': 1.0}
{'loss': 0.5564, 'grad_norm': 2.2205424308776855, 'learning_rate': 1.2498011679960007e-08, 'entropy': 0.5761293834075332, 'num_tokens':
 83149601.0, 'mean_token_accuracy': 0.8268802650272846, 'epoch': 1.0}
{'loss': 0.5312, 'grad_norm': 2.6596286296844482, 'learning_rate': 1.1361828799963643e-08, 'entropy': 0.5508037943392992, 'num_tokens':
 83166386.0, 'mean_token_accuracy': 0.8247018188238144, 'epoch': 1.0}
{'loss': 0.5234, 'grad_norm': 2.1411092281341553, 'learning_rate': 1.022564591996728e-08, 'entropy': 0.5520416988059879, 'num_tokens':
83183952.0, 'mean_token_accuracy': 0.8335484981536865, 'epoch': 1.0}
{'loss': 0.5497, 'grad_norm': 2.423283576965332, 'learning_rate': 9.089463039970914e-09, 'entropy': 0.5823555167764425, 'num_tokens': 8
3200428.0, 'mean_token_accuracy': 0.8191213145852089, 'epoch': 1.0}
{'loss': 0.5564, 'grad_norm': 2.0099804401397705, 'learning_rate': 7.95328015997455e-09, 'entropy': 0.5699185688048601, 'num_tokens': 8
3218990.0, 'mean_token_accuracy': 0.8298507906496525, 'epoch': 1.0}
{'loss': 0.5235, 'grad_norm': 1.9973183870315552, 'learning_rate': 6.817097279978186e-09, 'entropy': 0.551297714188695, 'num_tokens': 8
3236330.0, 'mean_token_accuracy': 0.8331429354846478, 'epoch': 1.0}
{'loss': 0.5467, 'grad_norm': 2.3594107627868652, 'learning_rate': 5.680914399981822e-09, 'entropy': 0.5545737722888588, 'num_tokens':
83254701.0, 'mean_token_accuracy': 0.8280885197222233, 'epoch': 1.0}
{'loss': 0.572, 'grad_norm': 2.245898962020874, 'learning_rate': 4.544731519985457e-09, 'entropy': 0.5783307179808617, 'num_tokens': 83
272339.0, 'mean_token_accuracy': 0.8227200582623482, 'epoch': 1.0}
{'loss': 0.578, 'grad_norm': 2.3958323001861572, 'learning_rate': 3.408548639989093e-09, 'entropy': 0.5950157677754759, 'num_tokens': 8
3287288.0, 'mean_token_accuracy': 0.8191494919359684, 'epoch': 1.0}
{'loss': 0.5632, 'grad_norm': 2.3703255653381348, 'learning_rate': 2.2723657599927285e-09, 'entropy': 0.5862594863399864, 'num_tokens':
 83304363.0, 'mean_token_accuracy': 0.8243992872536182, 'epoch': 1.0}
{'loss': 0.5549, 'grad_norm': 2.2291901111602783, 'learning_rate': 1.1361828799963642e-09, 'entropy': 0.5732654126361012, 'num_tokens':
 83320591.0, 'mean_token_accuracy': 0.8250299856066704, 'epoch': 1.0}
{'train_runtime': 67356.924, 'train_samples_per_second': 20.907, 'train_steps_per_second': 1.307, 'train_loss': 0.1620903710070505, 'en
tropy': 0.5838482262658291, 'num_tokens': 83333483.0, 'mean_token_accuracy': 0.8219176960773156, 'epoch': 1.0}
100%|██████████████████████████████████████████████████████████████████| 88014/88014 [18:42:36<00:00,  1.31it/s]

real    1123m9.762s
user    1044m57.858s
sys     74m44.848s
(lfm_env) ye@lst-hpc3090:~/exp/myMono_LFM$

In total 740m + 1123m.
```

Fig. Fine-tuning Log (about 31 hours)

Fine-tuning with Liquid Foundation Model

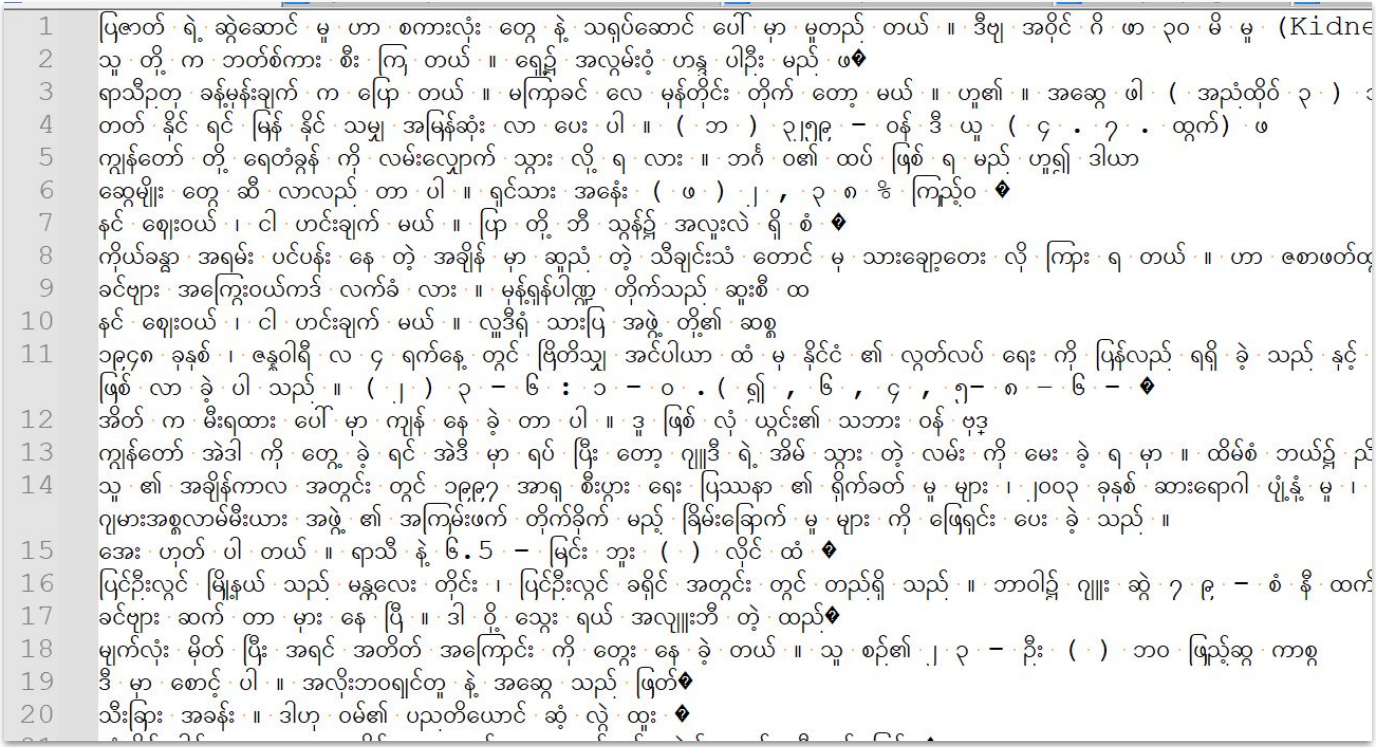


Fig. Generated text with fine-tuned model

- အစ စာကြောင်း အိုကေ
- ပုဒ်မ ရဲ့ နောက်မှာ ပြဿနာရှိ

Fine-tuning with Liquid Foundation Model

```
## Evaluation

(lfm_env) ye@lst-hpc3090:~/exp/myMono_LFM$ ./run_inference.sh
------------------------------------------------
 1. Running Perplexity Evaluation
------------------------------------------------
Loading tokenizer and model...
`torch_dtype` is deprecated! Use `dtype` instead!
Evaluating on otest.1k.word...
100%|██████████████████████████████████████████████████████████████████████████| 1000/1000 [00:27<00:00, 36.76it/s]

==============================
Evaluation Results
------------------------------
Average Loss: 0.9616
Perplexity:   2.6159
==============================

real    0m34.626s
user    0m37.286s
sys     0m0.640s
```

Fig. Perplexity for fine-tuned model

Fine-tuning with Liquid Foundation Model

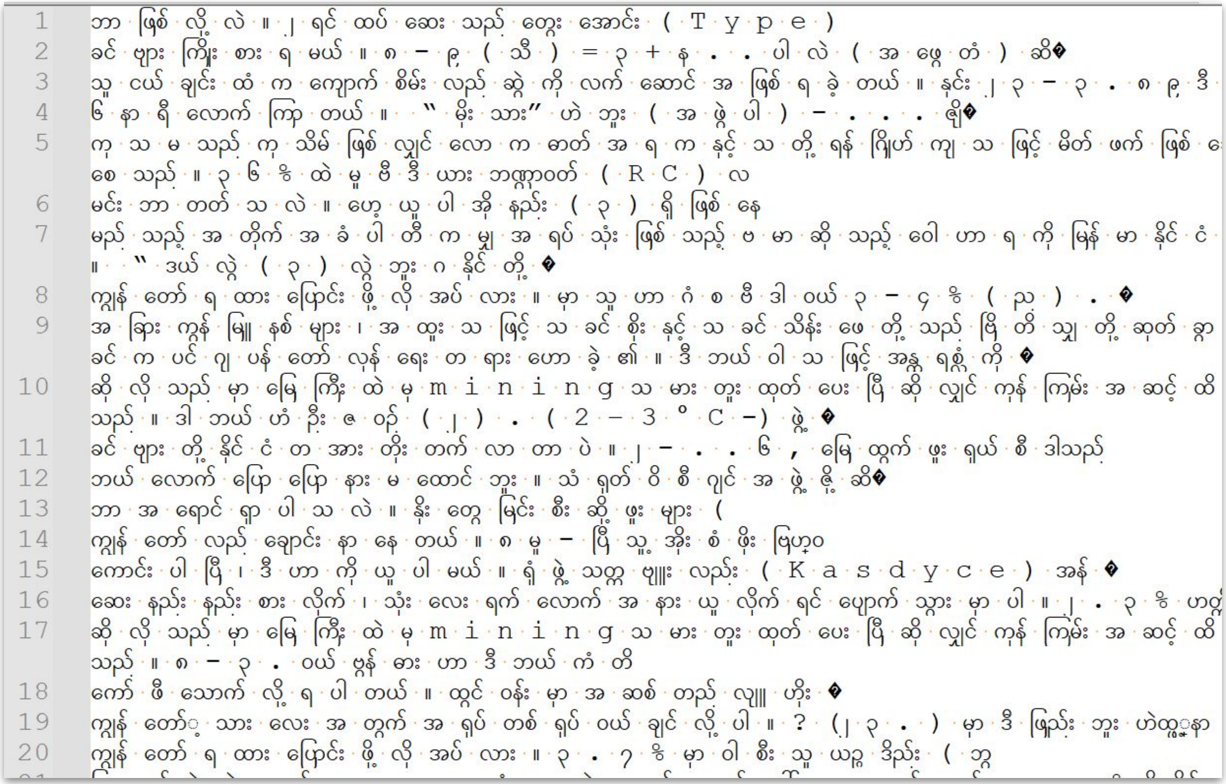


Fig. Generated text with syllable segmented monolingual corpus

Conclusion

- **An Emerging Frontier:** While LNNs are currently less mainstream than Transformers, they represent a significant shift toward more flexible, bio-inspired AI.
- **Continuous Real-Time Adaptation:** Unlike static models, LNNs adapt their parameters to changing data streams on the fly.
- **Optimized for the Edge:** Their compact footprint makes them ideal for low-power hardware, including drones, medical wearables, and autonomous vehicles.
- **Resilience to Noise:** LNNs excel at processing messy, real-world time-series data and remain stable even when encountering unfamiliar environments.
- **Transparent & Interpretable:** With fewer parameters and a structure rooted in differential equations, LNNs offer a clearer "window" into the model’s logic than traditional black-box architectures.
- **Sustainable & Green AI:** By drastically reducing computational overhead, LNNs provide a path toward high-performance AI with a minimal carbon footprint.

Conclusion

- **Limited Application Scope:** LNNs are primarily designed for time-series and sequential data, making them unsuitable for static data tasks like image classification.
- **Training Complexity:** Training LNNs involves complex differential equations, which can be computationally intensive to compute during the training phase.
- **Vanishing Gradient Problem:** Like many Recurrent Neural Networks (RNNs), LNNs can still struggle with learning long-term dependencies over very long sequences.
- **Newer Technology:** As a relatively new architecture (developed by MIT in 2020), there is less literature, tooling, and industry experience compared to traditional models.
- **Difficult Hyperparameter Tuning:** Finding the right parameters for the liquid layer can be difficult due to its dynamic nature.

References

1. OpenWorm: An open-source C. elegans nematode simulation: https://www.youtube.com/watch?v=Ejf6FwIibkE
2. The Whole Worm: Brain-Body-Environment Models of C. Elegans: https://www.youtube.com/watch?v=lXrtoqu4Uso
3. Map Making in C. elegans: Charting the Nervous System: https://www.youtube.com/watch?v=tWPXZ9tCRnk
4. OpenWorm C. elegans crawling simulation: https://www.youtube.com/watch?v=M0rpT831ngM
5. Stephen Larson - Open Worm: A community developed in silico model of C. Elegans (2014): https://www.youtube.com/watch?v=hhIe__5-6ck

References

6. Yemini E, Jucikas T, Grundy LJ, Brown AE, Schafer WR. A database of Caenorhabditis elegans behavioral phenotypes. Nat Methods. 2013 Sep;10(9):877-9. doi: 10.1038/nmeth.2560. Epub 2013 Jul 14. PMID: 23852451; PMCID: PMC3962822.
7. Hinrich Schulenburg, Marie-Anne Félix, The Natural Biotic Environment of Caenorhabditis elegans, Genetics, Volume 206, Issue 1, 1 May 2017, Pages 55–86, https://doi.org/10.1534/genetics.116.195511
8. WormAtlas: https://www.wormatlas.org/index.html
9. An online database for behavioural phenotypes of the nematode Caenorhabditis elegans, Tadas Jucikas, Darwin College, 15 Aug 2013, https://www2.mrc-lmb.cam.ac.uk/groups/wschafer/wp-content/uploads/sites/38/tjucikas_phd_thesis.pdf
10. A community of Japanese researchers who study or are interested in the nematode studies: https://plaza.umin.ac.jp/wormjp/wordpress/information-for-researchers/#information-for-researchers_con2

References

11. International Worm Meeting: https://genetics-gsa.org/celegans/
12. Your C. Elegans Meetings Headquarters: https://wormmeetings.weebly.com/
13. napari: a fast, interactive viewer for multi-dimensional images in Python: https://napari.org/stable/
14. https://github.com/royerlab/inTRACKtive/blob/main/python/src/intracktive/examples/notebook2_inTRACKtive_from_napari.ipynb
15. C. elegans 101: A Guide to Picking C. elegans: https://www.youtube.com/watch?v=5ipoAwukPA8
16. OpenWorm, An open-source project dedicated to creating a virtual C. elegans nematode in a computer: https://github.com/openworm

References

17. The Worm Wide Web – Max Perutz Science Writing Award: https://mrclmb.ac.uk/news-events/articles/the-worm-wide-web-max-perutz-science-writing-award/
18. drone_causality: https://github.com/makramchahine/drone_causality
19. Rosetta Drone: https://github.com/makramchahine/rosetta_drone
20. Chahine, M., Hasani, R.M., Kao, P., Ray, A., Shubert, R., Lechner, M., Amini, A., & Rus, D. (2023). Robust flight navigation out of distribution with liquid neural networks. Science Robotics, 8.
21. Adaptable Aviators: future of autonomous navigation Liquid networks | Ramin Hasani | TEDxMIT Salon: https://www.youtube.com/watch?v=2BoxhZUi01Y

References

22. Liquid Neural Networks for Autonomous Driving: https://www.youtube.com/watch?v=ql3ETcRDMEM
23. Liquid Neural Networks: https://www.youtube.com/watch?v=IlliqYiRhMU
24. Panel Presentation: Liquid Neural Networks with Live Q&A: https://www.youtube.com/watch?v=HGLjOxQxcr0
25. Liquid AI's Ramin Hasani on liquid neural networks, AI advancement, the race to AGI & more! | E1928: https://www.youtube.com/watch?v=iXQRhDeOnV4
26. Building More Efficient AI: How Liquid AI Is Working To Optimize AI: https://www.youtube.com/watch?v=kYAg-o_PIcM
27. CEO of Liquid AI Ramin Hasani Says a Worm is Changing the Future of AI: https://www.youtube.com/watch?v=_T2MUuIIYw0

References

28. Liquid AI: Product Launch Webcast 10/23: https://www.youtube.com/watch?v=d19jhYtwgCA
29. Liquid Neural Networks, A New Idea That Allows AI To Learn Even After Training: https://www.youtube.com/watch?v=0FNkrjVIcuk
30. Liquid AI: MIT's $2B LLM Startup | Agentic EP 10: https://www.youtube.com/watch?v=XyNRKU5KVqM
31. New “Liquid” Model - Benchmarks Are Useless: https://www.youtube.com/watch?v=QzNI1HRIpYo
32. PhD Thesis, Interpretable Recurrent Neural Networks in Continuous-time Control Environments, Vienna PhD School of Informatics by Ramin Hasani, Registration Number 01529745, 10 April 2020, https://repositum.tuwien.at/bitstream/20.500.12708/1068/2/Hasani%20Ramin%20-%202020%20-%20Interpretable%20recurrent%20neural%20networks%20in%20continuous-time...pdf

References

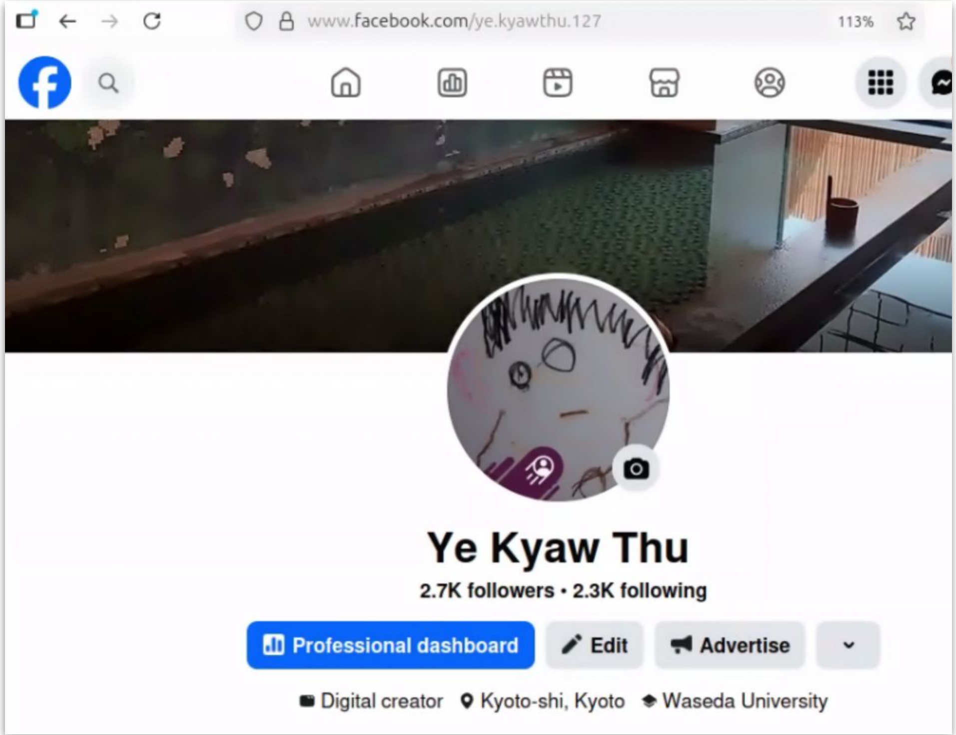
33. CfC (TensorFlow/PyTorch): https://github.com/raminmh/CfC
34. LTC (Original): https://github.com/raminmh/liquid_time_constant_networks
35. Liquid-S4: https://github.com/raminmh/liquid-s4
36. NCPs (Includes LTC/CfC examples): https://github.com/mlech26l/keras-ncp
37. LiqudNet (PyTorch): https://github.com/kyegomez/LiqudNet
38. lnnrs (Rust WIP): https://github.com/postrv/lnnrs
39. First Complete Nervous System Wiring Map of an Animal: Why You Should Care about the Worm Connectome: https://www.youtube.com/watch?v=LBKEyGaPamY
40. Looking for AGI? Try C. elegans, Not ChatGPT: https://decidingbetter.com/looking-for-agi-try-c-elegans-not-chatgpt/

References

41. C. elegans Neural Network details by nikhil bhatla: http://wormweb.org/neuralnet#c=BAG&m=1
42. Biologically-inspired Neural Networks for Self-Driving Cars: https://www.youtube.com/watch?v=8KBOf7NJh4Y

Thank you!

Homepage: https://sites.google.com/site/yekyawthunlp/
GitHub: https://github.com/ye-kyaw-thu
Email: ykt.nlp.ai@gmail.com



- မတ်လထဲမှာ AI Engineering (Foundation) အတန်းကို ဖွင့်ပါမယ်
- Course နဲ့ ပတ်သက်တဲ့ အသေးစိတ်က Facebook မှာပဲ ဝင်လေ့လာပါ

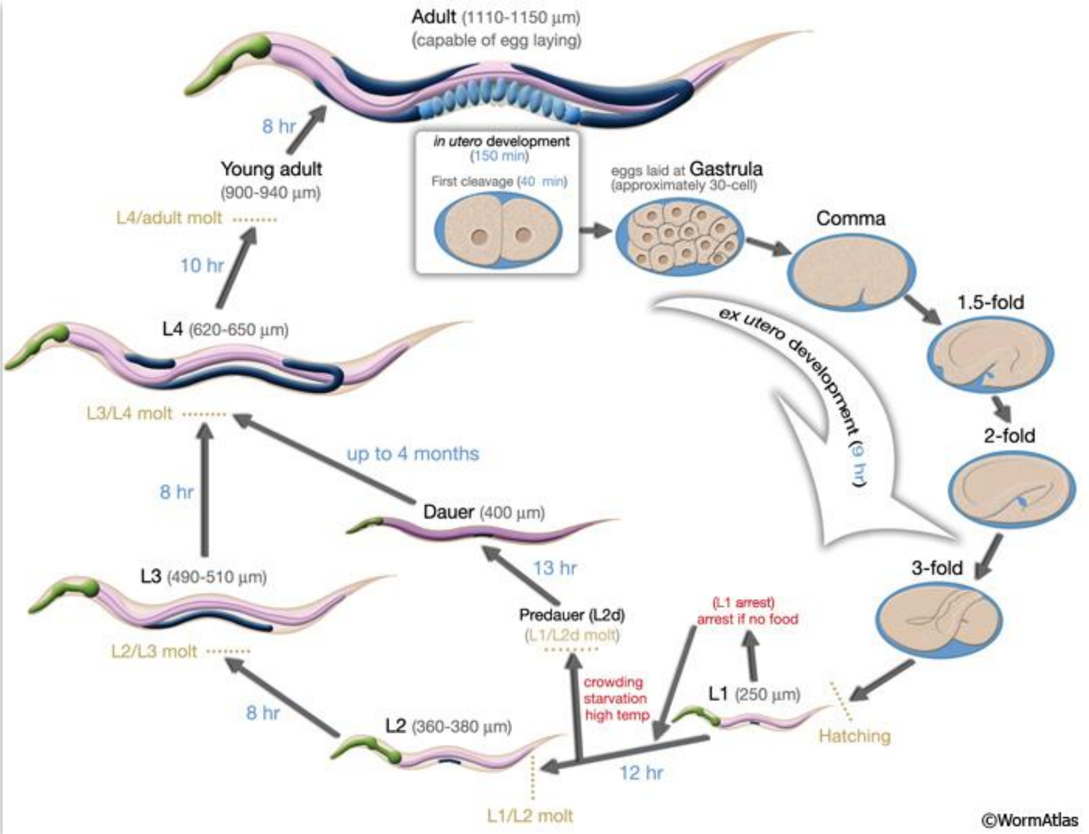
Lifespan of C. elegans

- Average lifespan: about 2–3 weeks
- Time to adulthood: ~3 days (from egg to adult)
- Reproductive period: days 3–10
- Old age / decline: after ~10 days

Lifespan of C. elegans

- C. elegans reproduces using hermaphrodites and males, with no females; hermaphrodites can self-fertilize or mate with males.
- A hermaphrodite has:
  - Male reproductive organs (produces sperm)
  - Female reproductive organs (produces eggs)
- A single hermaphrodite can:
  - Reproduce by itself (self-fertilization)
  - Or mate with a male
- Males make up only ~0.1–0.2% of wild populations, they arise due to chromosomal differences or errors, they exist mainly to increase genetic diversity

Lifespan of C. elegans



- C. elegans life cycle with dauer branch (20 C) from egg to reaching L4 (47 hours)

Sources:
https://www.wormatlas.org/hermaphrodite/introduction/mainframe.htm

https://evandrofanglab.com/wp-content/uploads/2021/07/2021-07-08-how-to-work-in-the-lab-handing-c.-elegans-c.-elegans-other-details.pdf

10 Reasons Why C. elegans is Essential to Science

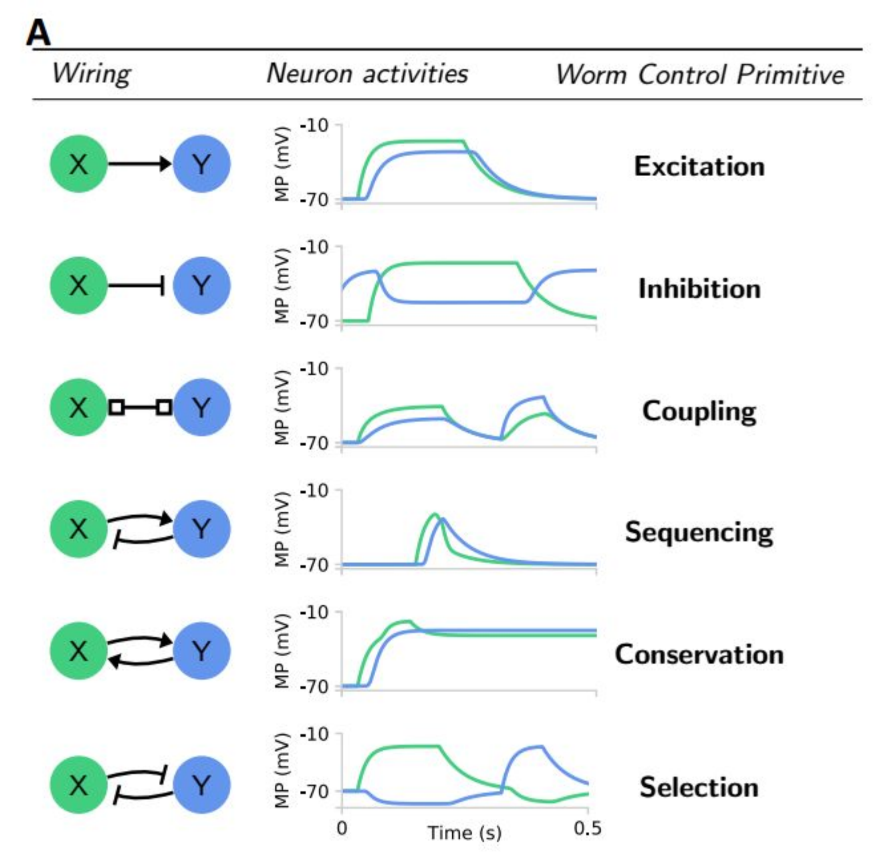
- Extreme Resilience: They can survive catastrophic events and extreme environments, such as the disintegration of a space shuttle during re-entry, making them ideal for space-based research.
- Genetic Pioneer: C. elegans was the first animal in history to have its entire genome sequenced, providing a foundational template for understanding animal genetics.
- Proven Scientific Value: Research involving these worms has been so impactful that it has contributed to three Nobel Prizes.
- Manageable Complexity: While the human brain has 100 billion neurons, the worm has only about 300, making it a "simplified" model that is much easier for scientists to map and study in its entirety.
- Advanced Behaviors: Despite their small size, they exhibit complex behaviors—such as foraging, escaping threats, forming memories, and sensing magnetic fields—that mirror higher-order organisms.

10 Reasons Why C. elegans is Essential to Science

- The Complete Connectome: It is currently the only animal with a fully detailed map of its neural wiring (the "connectome"), a task that remains impossible for larger brains like those of mice or humans.
- Shared Genetics: Humans and C. elegans share significant genetic similarities, meaning discoveries made in the worm's brain can often be translated to help us understand the human brain.
- Chemical Network Mapping: They allow scientists to study "invisible" chemical networks (monoamines like dopamine and serotonin) which are crucial to understanding how brains regulate information and mood.
- Insights into Mental Health: By studying how monoamines coordinate in the worm, researchers can better understand the biological roots of human psychiatric conditions like depression and schizophrenia.
- Modeling Human Disorders: The worm exhibits behaviors that mirror human drug dependence and eating disorders, providing a unique opportunity to test theories on addiction and behavior in a controlled environment.

Adaptable Aviators: future of autonomous navigation Liquid networks | Ramin Hasani | TEDxMIT Salon

Operators



- Design Operators. (A) Time series of the activity of DOs.
- From PhD thesis, Interpretable Recurrent Neural Networks in Continuous-time Control Environments, Vienna PhD School of Informatics by Ramin Hasani, Registration Number 01529745, 10 April 2020

Sources:
https://repositum.tuwien.at/bitstream/20.500.12708/1068/2/Hasani%20Ramin%20-%202020%20-%20Interpretable%20recurrent%20neural%20networks%20in%20continuous-time...pdf